

Chương 3:

TIẾN TRÌNH VÀ LUỒNG

ThS. GVC Tô Oai Hùng

1

1

Nội Dung

- 1. Tiến trình.**
- 2. Luồng.**
- 3. Truyền thông giữa các tiến trình.**
- 4. Đồng bộ hóa tiến trình.**
- 5. Điều phối tiến trình.**

ThS. GVC Tô Oai Hùng

2

2

3.1 Tiến Trình

1. Mô hình.
2. Hiện thực.

ThS. GVC Tô Oai Hùng

3

3

3.1.1 Mô hình Tiến Trình

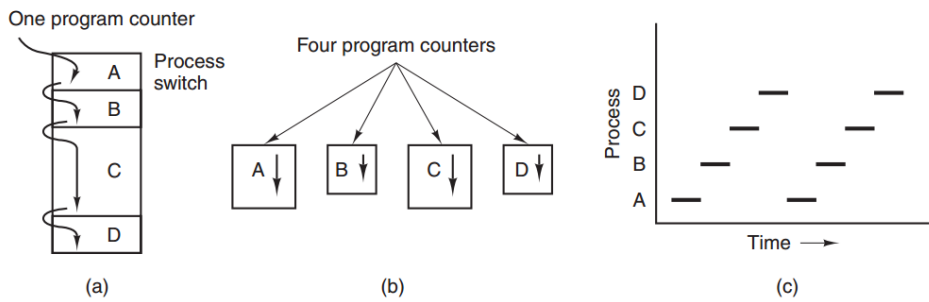
- ❖ Chương trình phần mềm khi thực thi phải được nạp vào bộ nhớ chính.
- ❖ CPU đọc các chỉ thị từ bộ nhớ chính để xử lý.
- ❖ Hệ thống máy tính hiện đại cho phép nhiều chương trình được nạp vào bộ nhớ và thực hiện đồng thời → cần có cơ chế kiểm soát hoạt động của các chương trình khác nhau → khái niệm tiến trình (*process*).

ThS. GVC Tô Oai Hùng

4

4

Mô hình Tiến Trình



(a): A, B, C, D thực thi tuần tự nên chỉ cần 1 bộ đếm chương trình.

(b): A, B, C, D thực thi đồng thời bằng cách chia sẻ CPU và phải sử dụng 4 bộ đếm chương trình.

(c): Mỗi thời điểm chỉ có 1 tiến trình thực thi.

ThS. GVC Tô Oai Hùng

5

5

Mô hình Tiến Trình

❖ Tiến trình là một chương trình đang được thực thi.

❖ Một tiến trình cần sử dụng các tài nguyên:

- CPU.
- Bộ nhớ.
- Tập tin.
- Thiết bị nhập/ xuất.
- ...

Task Manager				
File Options View				
Processes Performance App history Startup Users Details Services				
Name	Status	26% CPU	40% Memory	3% Disk
Apps (5)				
> Foxit Reader 7.0, Best Reader for...		3.2%	46.6 MB	0 MB/s
> Google Chrome (5)		0.6%	527.6 MB	0 MB/s
> Microsoft PowerPoint		12.7%	279.0 MB	0 MB/s
> Task Manager		0.4%	27.9 MB	0 MB/s
> Windows Explorer		0.9%	53.8 MB	0 MB/s
Background processes (123)				
> Adobe Genuine Software Integr...		0%	2.5 MB	0 MB/s
> Adobe Genuine Software Servic...		0%	2.8 MB	0 MB/s
> Adobe Update Service (32 bit)		0%	0.7 MB	0 MB/s
AlpsAlpine Pointing-device Driver		0%	0.6 MB	0 MB/s
AlpsAlpine Pointing-device Driver		0%	1.9 MB	0 MB/s
AlpsAlpine Pointing-device Driv...		0%	0.6 MB	0 MB/s

ThS. GVC Tô Oai Hùng

6

Mô hình Tiến Trình

- ❖ Một tiến trình bao gồm:
 - Text section (program code), data section (chứa global variables)
 - program counter (PC), process status word (PSW), stack pointer (SP), memory management registers,...
- ❖ Phân biệt chương trình và tiến trình:
 - Chương trình: là một thực thể thụ động (tập tin có chứa một danh sách các lệnh được lưu trữ trên đĩa - file thực thi)
 - Tiến trình: là một thực thể hoạt động, với một bộ đếm chương trình xác định các chỉ lệnh kế tiếp để thực hiện và một số tài nguyên liên quan.

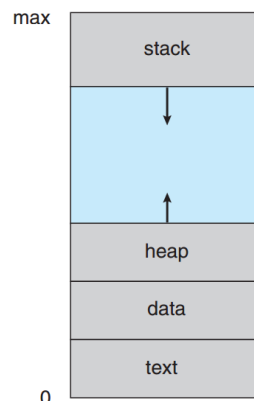
ThS. GVC Tô Oai Hùng

7

7

Mô hình Tiến Trình

- Một chương trình sẽ trở thành một tiến trình khi một tập tin thực thi được nạp vào bộ nhớ.
- ❖ Tiến trình trong bộ nhớ:



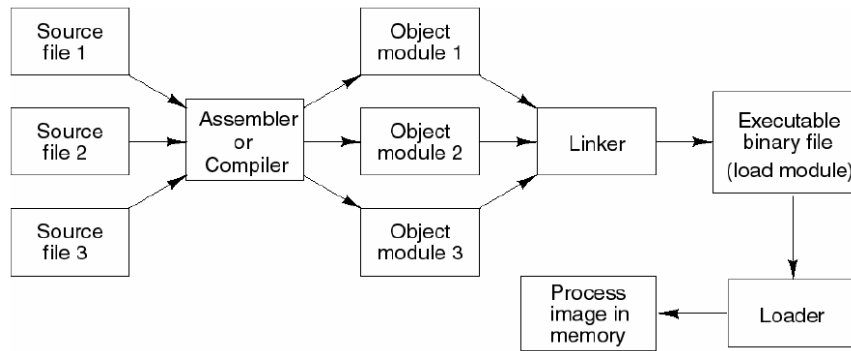
ThS. GVC Tô Oai Hùng

8

8

Mô hình Tiến Trình

❖ Các bước nạp chương trình vào bộ nhớ:



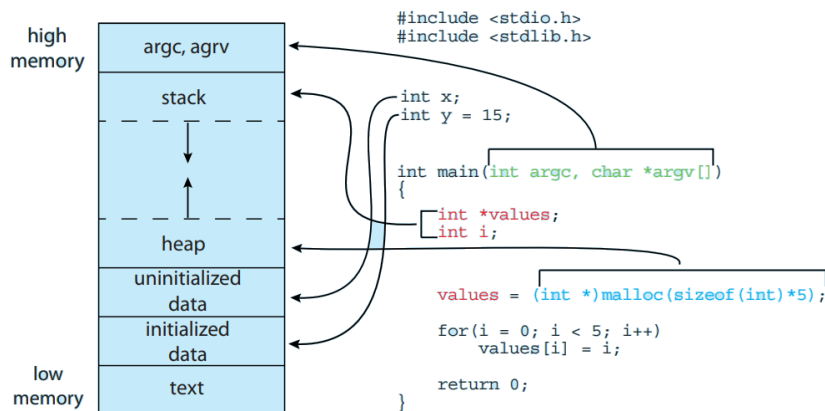
ThS. GVC Tô Oai Hùng

9

9

Mô hình Tiến Trình

❖ Ví dụ bố trí bộ nhớ cho chương trình C:



ThS. GVC Tô Oai Hùng

10

10

3.1.2 Hiện Thực Tiến Trình

1. Vai trò của hệ điều hành trong quản lý tiến trình.
2. Tạo tiến trình.
3. Các trạng thái của tiến trình.
4. Các thao tác với tiến trình .
5. Thực thi các tiến trình.
6. PCB (*Process Control Block*).

ThS. GVC Tô Oai Hùng

11

11

Vai Trò của HĐH Trong Quản Lý Tiến Trình

- ❖ Tạo và hủy tiến trình của người dùng và tiến trình hệ thống.
- ❖ Dừng, khôi phục (*resume*) tiến trình.
- ❖ Cung cấp các cơ chế để đồng bộ hóa tiến trình.
- ❖ Cung cấp các cơ chế liên lạc giữa các tiến trình.
- ❖ Cung cấp cơ chế xử lý bế tắc (*deadlock*).

ThS. GVC Tô Oai Hùng

12

12

Tạo Tiến Trình

- ❖ Các bước hệ điều hành khởi tạo tiến trình:
 - Cấp phát định danh duy nhất cho tiến trình (*process number, process identifier, pid*).
 - Cấp phát không gian nhớ để nạp tiến trình.
 - Khởi tạo khối dữ liệu Process Control Block (PCB) lưu các thông tin về tiến trình được tạo.
 - Thiết lập các mối liên hệ cần thiết (vd: sắp PCB vào hàng đợi định thời, ...).

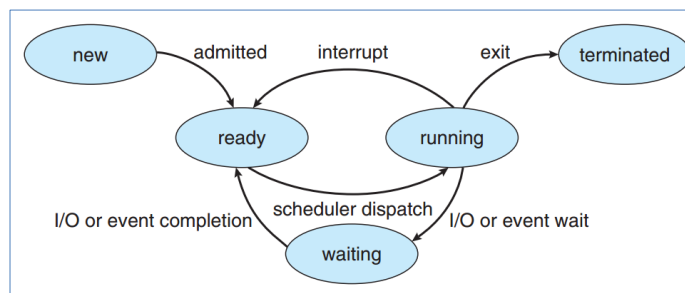
ThS. GVC Tô Oai Hùng

13

13

Các Trạng Thái của Tiến Trình

- ❖ New: Tiến trình đang được tạo lập.
- ❖ Running: các chỉ thị của tiến trình đang được xử lý.
- ❖ Waiting: Tiến trình chờ được cấp phát một tài nguyên, hay chờ một sự kiện xảy ra.
- ❖ Ready: Tiến trình chờ được cấp phát CPU.
- ❖ Terminated: Tiến trình kết thúc xử lý.

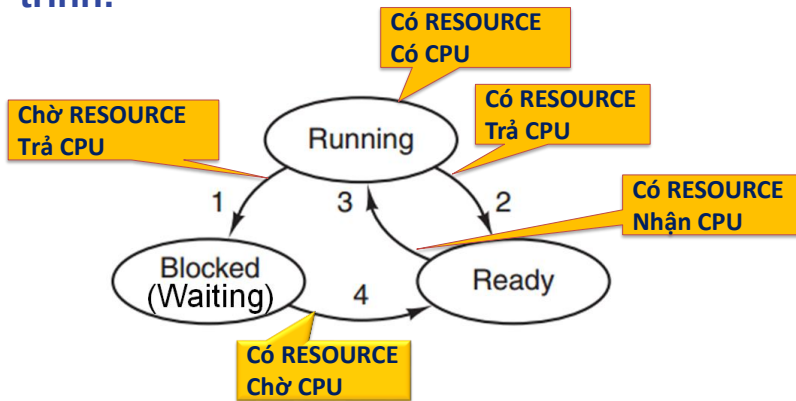


14

14

Các Trạng Thái của Tiến Trình

- ❖ Quá trình chuyển đổi trạng thái của tiến trình:



ThS. GVC Tô Oai Hùng

15

15

Các Trạng Thái của Tiến Trình

- ❖ Ví dụ về quá trình chuyển đổi trạng thái của tiến trình:

- Cho đoạn chương trình C trên Linux.
- Biên dịch:

```
gcc test.c -o test
```

- Thực thi chương trình test
- ```
./test
```

- Tiến trình test được tạo ra, thực thi và kết thúc.

- Trạng thái của tiến trình Test:

- new → ready → running → waiting (do chờ I/O khi gọi printf) → ready → running → terminated.

```
/* test.c */
int main(int argc,
char** argv)
{
 printf("Hello \n");
 exit (0);
}
```

ThS. GVC Tô Oai Hùng

16

16



## Các Thao Tác Với Tiến Trình

- ❖ Tiến trình được tạo khi:
  - Hệ thống khởi động.
  - Một tiến trình đang chạy thực hiện lời gọi hệ thống tạo tiến trình mới.
  - Người dùng tạo.
  - Bắt đầu một batch job.

ThS. GVC Tô Oai Hùng

17

17

## Các Thao Tác Với Tiến Trình

- ❖ Tiến trình mới:
  - Nhận tài nguyên từ HĐH hoặc từ tiến trình cha.
  - Tài nguyên: nếu tiến trình con được tạo từ tiến trình cha:
    - Tiến trình cha và con chia sẻ mọi tài nguyên.
    - Tiến trình con chỉ chia sẻ một phần tài nguyên của cha.
  - Trình tự thực thi:
    - Tiến trình cha và con thực thi đồng thời (*concurrently*).
    - Tiến trình cha đợi đến khi các tiến trình con kết thúc.

ThS. GVC Tô Oai Hùng

18

18

## Các Thao Tác Với Tiến Trình

- Không gian địa chỉ (*address space*):
  - Không gian địa chỉ của tiến trình con được nhân bản từ cha.
  - Không gian địa chỉ của tiến trình con được khởi tạo từ template.

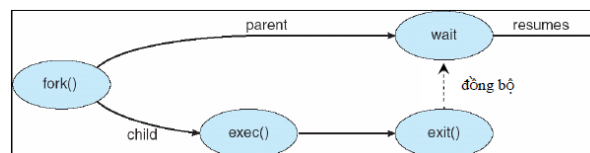
ThS. GVC Tô Oai Hùng

19

19

## Các Thao Tác Với Tiến Trình

```
#include <stdio.h>
#include <unistd.h>
int main (int argc, char *argv[]){
 int pid;
 pid = fork();
 if (pid > 0) {
 printf("This is parent process");
 wait (NULL);
 exit(0);
 }
 else if (pid == 0) {
 printf("This is child process");
 execlp("/bin/ls", "ls", NULL);
 exit(0);
 }
 else {
 printf("Fork error\n");
 exit(-1);
 }
}
```



ThS. GVC Tô Oai Hùng

20

## Các Thao Tác Với Tiến Trình

### ❖ Kết thúc tiến trình:

- Tiến trình kết thúc trong các trường hợp:
  - Tiến trình tự kết thúc bình thường: tiến trình kết thúc khi thực thi lệnh cuối và gọi exit
  - Tiến trình bị lỗi và tự kết thúc (do trình biên dịch xử lý)
  - Tiến trình bị lỗi và buộc phải kết thúc do phần cứng (ví dụ thực hiện một lệnh không hợp lệ, tham chiếu đến bộ nhớ không tồn tại, lỗi chia cho không,...) → phát sinh một ngắt (*interrupted*).

ThS. GVC Tô Oai Hùng

21

21

## Các Thao Tác Với Tiến Trình

- Tiến trình kết thúc do tiến trình khác (ví dụ tiến trình cha).
  - + Gọi abort với tham số là pid của tiến trình cần được kết thúc.
- Khi tiến trình kết thúc, HĐH thu hồi tất cả các tài nguyên của tiến trình.

ThS. GVC Tô Oai Hùng

22

22

## Thực Thi Các Tiến Trình

- ❖ Để thực thi được mô hình tiến trình, HĐH duy trì một bảng gọi là bảng tiến trình.
- ❖ Mỗi phần tử trong bảng là một khối quản lý tiến trình (*Process Control Block* - PCB).
- ❖ Mỗi PCB quản lý một tiến trình.
- ❖ Mỗi tiến trình khi được tạo sẽ được cấp phát một (PCB).

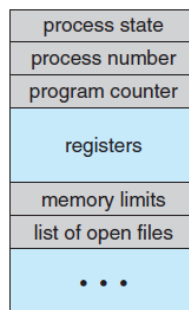
ThS. GVC Tô Oai Hùng

23

23

## PCB – Process Control Block

- ❖ Là một cấu trúc dữ liệu lưu các thông tin:
  - Định danh (*Process Number*).
  - Trạng thái tiến trình (*Process State*).
  - Bộ đếm chương trình (*Program counter*).
  - Các thanh ghi (*CPU registers*).



ThS. GVC Tô Oai Hùng

Process control block (PCB).

24

24

## PCB

- ❖ Là một cấu trúc dữ liệu lưu các thông tin:
  - Thông tin lập thời biểu CPU: Độ ưu tiên, con trỏ đến hàng đợi, ...
  - Thông tin quản lý bộ nhớ.
  - Thông tin tài khoản: Lượng CPU, thời gian sử dụng, ...
  - Thông tin trạng thái I/O.

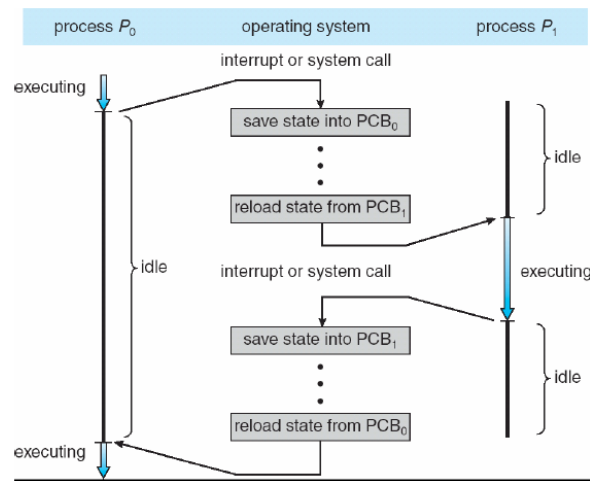
ThS. GVC Tô Oai Hùng

25

25

## PCB

- ❖ Sơ đồ chuyển CPU giữa các tiến trình:



ThS. GVC Tô Oai Hùng

26

26

## 3.2 Luồng

1. Mô hình.
2. Hiện thực.

ThS. GVC Tô Oai Hùng

27

27

### 3.2.1 Mô Hình Luồng

- ❖ Nhìn lại giải pháp đa tiến trình:
  - Các tiến trình độc lập, không có sự liên lạc với nhau.
  - Mỗi tiến trình có một không gian địa chỉ và một dòng xử lý duy nhất → tiến trình thực hiện chỉ có một tác vụ tại một thời điểm.
  - Muốn trao đổi thông tin với nhau, các chương trình cần được xây dựng theo mô hình liên lạc đa tiến trình (IPC – *Inter-Process Communication*) → Phức tạp, chi phí cao.

ThS. GVC Tô Oai Hùng

28

28

## Mô Hình Luồng

- ❖ Để tăng tốc độ và sử dụng CPU hiệu quả hơn:
  - Cần nhiều dòng xử lý cùng chia sẻ một không gian địa chỉ.
  - Các dòng xử lý hoạt động song song tương tự như tiến trình phân biệt.
  - Mỗi dòng xử lý được gọi là một luồng (*thread*).
- ❖ Hầu hết các HĐH hiện đại đều được thực hiện theo mô hình luồng.
- ❖ Ví dụ:
  - Ứng dụng Word.
  - Các xử lý trên máy chủ Web.

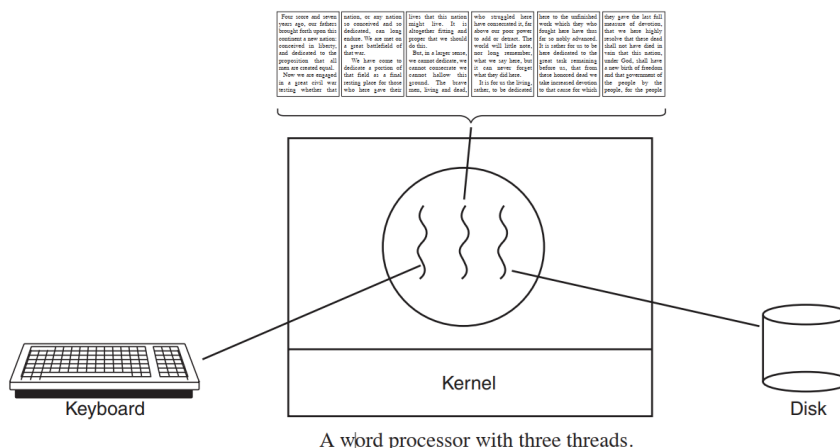
ThS. GVC Tô Oai Hùng

29

29

## Mô Hình Luồng

- ❖ Ứng dụng Word với ba luồng xử lý:



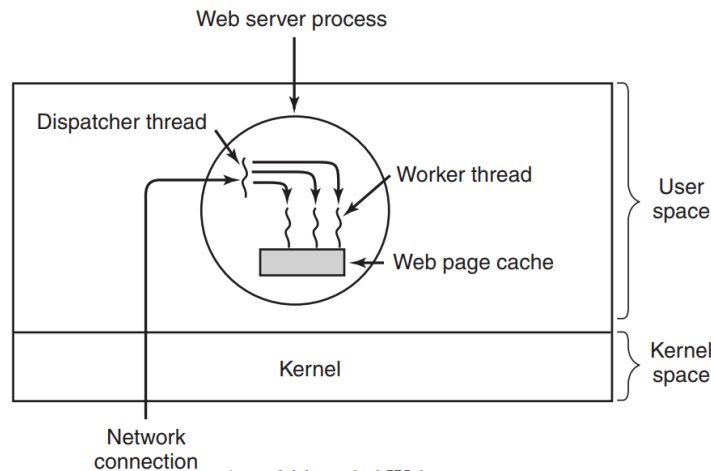
ThS. GVC Tô Oai Hùng

30

30

## Mô Hình Luồng

### ❖ Web server xử lý đa luồng:



A multithreaded Web server.

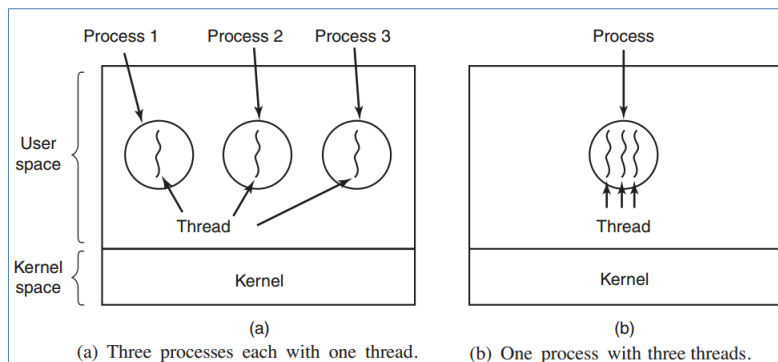
ThS. GVC Tô Oai Hùng

31

31

## Mô Hình Luồng

### ❖ Đơn luồng và đa luồng:



- Trong hình (a), mỗi luồng hoạt động trong một không gian địa chỉ khác nhau.
- Trong hình (b), cả ba luồng đều có chung không gian địa chỉ.

ThS. GVC Tô Oai Hùng

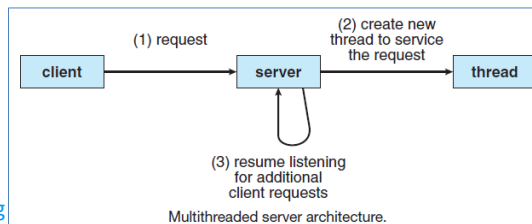
32

32



## Mô Hình Luồng

- ❖ Luồng là một dòng xử lý trong một tiến trình.
- ❖ Mỗi tiến trình luôn có một luồng chính (dòng xử lý cho hàm `main()`).
- ❖ Ngoài luồng chính, tiến trình còn có thể có nhiều luồng con khác.
- ❖ Các luồng của một tiến trình:
  - Chia sẻ không gian vùng code và data.
  - Có vùng stack riêng.



ThS. GVC Tô Oai Hùng

33

33

## Mô Hình Luồng

- ❖ Các trạng thái của luồng:
  - **Spawn:** Luồng được tạo.
    - Khi một tiến trình mới được tạo, một luồng chính cũng được tạo ra.
    - Một luồng trong tiến trình có thể tạo ra một luồng khác, cung cấp con trỏ lệnh, các đối số, thanh ghi ngữ cảnh, không gian stack và luồng mới tạo được đưa vào ready queue.
  - **Blocked:**
    - Khi một luồng phải chờ một sự kiện → blocked (lưu các thanh ghi người dùng, bộ đếm chương trình, con trỏ stack).

ThS. GVC Tô Oai Hùng

34

34

## Mô Hình Luồng

- CPU có thể chuyển sang thực thi một luồng khác trong cùng tiến trình hoặc tiến trình khác.
- Unblock: Khi sự kiện mà luồng đang chờ đã sẵn sàng, luồng chuyển sang trạng thái ready (được đưa vào ready queue).
- Finish: Khi luồng kết thúc.

ThS. GVC Tô Oai Hùng

35

35

## Mô Hình Luồng

### ❖ Ưu điểm của luồng:

- Tính đáp ứng (*responsiveness*): chương trình tiếp tục chạy nếu một phần của nó bị blocked hay đang thực thi một thao tác dài → đáp ứng nhanh đến người sử dụng, đặc biệt trong các hệ thống chia sẻ thời gian thực.
- Chia sẻ tài nguyên (*resource sharing*): các luồng chia sẻ bộ nhớ và tài nguyên của tiến trình, và các luồng có liên quan (luồng tạo ra nó) → cho phép một ứng dụng có nhiều luồng khác nhau hoạt động trong cùng một không gian địa chỉ.

ThS. GVC Tô Oai Hùng

36

36

## Mô Hình Luồng

- **Tính kinh tế (economy):**
  - Cấp phát bộ nhớ và tài nguyên cho việc tạo tiến trình: Tốn kém.
  - Luồng chia sẻ tài nguyên của tiến trình → tạo luồng và chuyển ngữ cảnh ít tốn thời gian (thấp 5 lần so với tiến trình).
  - Thời gian tạo tiến trình gấp 30 lần so với tạo luồng.
- **Khả năng mở rộng (scalability):** Tận dụng được kiến trúc đa xử lý (Utilization of multiprocessor architecture):
  - Luồng có thể chạy song song trên các CPU khác nhau.
  - Đa luồng trên máy nhiều CPU làm tăng tính đồng thời.

ThS. GVC Tô Oai Hùng

37

37

## 3.2.2 Hiện Thực Luồng

- ❖ Phân loại luồng.
- ❖ Mô hình đa luồng.
- ❖ Khối quản lý luồng.
- ❖ Tạo luồng.
- ❖ So sánh luồng với tiến trình.

ThS. GVC Tô Oai Hùng

38

38

## Phân Loại Luồng

### ❖ User-Level Thread (ULT) – Luồng mức người dùng:

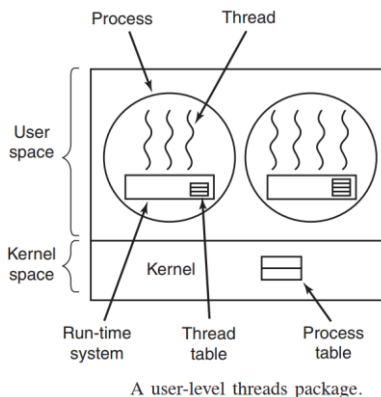
- Việc quản lý các luồng được thực hiện bởi ứng dụng.
- Gói các luồng được đặt hoàn toàn vào không gian người dùng.
- Nhân không biết gì về sự tồn tại của luồng.
- Bất kỳ ứng dụng nào cũng có thể được lập trình đa luồng bằng cách sử dụng thư viện luồng, là một thư viện các quy trình để quản lý ULT. Ví dụ:

```
public class ThreadDemo extends Thread
```

ThS. GVC Tô Oai Hùng

39

## Phân Loại Luồng



ThS. GVC Tô Oai Hùng

- Mỗi tiến trình cần có bảng luồng (*thread table*) để theo dõi các luồng trong tiến trình đó.
- Vai trò của thư viện luồng:
  - Khởi tạo, định thời và quản lý luồng.
  - Truyền thông giữa các luồng.
  - Lưu giữ và khôi phục ngữ cảnh của luồng.
  - Không cần hỗ trợ từ kernel.

40

40

## Phân Loại Luồng

- Ưu điểm: Tạo và quản lý nhanh.
- Nhược điểm:
  - Không tận dụng kiến trúc nhiều CPU: hai luồng của một tác vụ không thể chạy trên hai CPU.
- Một số thư viện luồng người dùng:
  - POSIX pthreads.
  - Mach C-threads.
  - Solaris UI-threads.

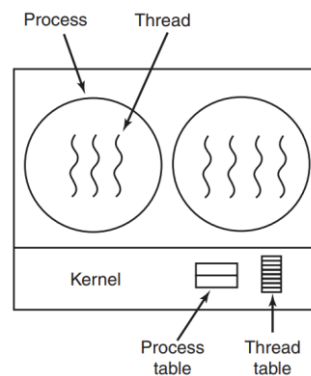
ThS. GVC Tô Oai Hùng

41

41

## Phân Loại Luồng

- ❖ Kernel-Level Thread – Luồng mức nhân/hệ thống:
  - Nhân duy trì thông tin về các tiến trình và luồng bằng các bảng Process table và Thread table.



ThS. GVC Tô Oai Hùng

A threads package managed by the kernel.

42

42

## Phân Loại Luồng

- Việc quản lý luồng được thực hiện bởi nhân. Không tồn tại mã quản lý luồng trong ứng dụng.
- Nhân thực hiện việc tạo luồng, định thời và quản lý trong không gian của nhân.
- Nhược điểm: Bởi vì việc quản lý luồng được thực hiện bởi hệ điều hành, luồng mức nhân được tạo và quản lý chậm hơn luồng mức người dùng.
- Ưu điểm: Nếu một luồng bị blocked → có thể chuyển một luồng khác trong ứng dụng để thực thi → trong môi trường nhiều CPU, nhân có thể định thời cho luồng trên các CPU khác nhau.

ThS. GVC Tô Oai Hùng

43

43

## Phân Loại Luồng

- ❖ Nhiều hệ thống hỗ trợ cả luồng mức người dùng và luồng mức nhân để tạo nhiều mô hình đa luồng khác nhau.
- ❖ Việc tạo luồng được thực hiện trong không gian người dùng.

ThS. GVC Tô Oai Hùng

44

44







## So Sánh Luồng với Tiến Trình

| Tiến trình                                                                                                                        | Luồng                                                                                             |
|-----------------------------------------------------------------------------------------------------------------------------------|---------------------------------------------------------------------------------------------------|
| Các tác vụ điều hành tiến trình tốn nhiều tài nguyên.                                                                             | Các tác vụ điều hành luồng ít tốn chi phí thực hiện hơn so với tiến trình.                        |
| Tiến trình chuyển đổi cần tương tác với hệ điều hành.                                                                             | Liên lạc giữa các luồng thông qua chia sẻ bộ nhớ, không cần sự can thiệp của kernel.              |
| Trong nhiều môi trường xử lý, mỗi tiến trình thực thi có nguồn tài nguyên bộ nhớ và tập tin riêng của mình.                       | Tất cả các luồng có thể chia sẻ các tập tin mở, tiến trình con.                                   |
| Nếu một tiến trình bị khóa (blocked), không có tiến trình nào khác có thể thực thi cho đến khi tiến trình ban đầu được unblocked. | Khi một tiến trình bị khóa và chờ, luồng kế tiếp trong cùng một task có thể chạy.                 |
| Nhiều tiến trình hoạt động độc lập với các tiến trình khác.                                                                       | Một luồng có thể đọc, ghi, thay đổi dữ liệu của luồng khác. <span style="float: right;">49</span> |

49

### 3.3. Truyền Thông Giữa Các Tiến Trình

#### ❖ Mục tiêu:

- Chia sẻ thông tin.
- Hợp tác hoàn thành tác vụ:
  - Chia nhỏ tác vụ để có thể thực thi song song.
  - Dữ liệu ra của tiến trình này là dữ liệu đầu vào cho tiến trình khác.
- HĐH cần cung cấp cơ chế để các tiến trình có thể trao đổi thông tin với nhau.

ThS. GVC Tô Oai Hùng

50

50

### 3.3.1 Các Dạng Tương Tác Giữa Các Tiến Trình

1. Tín hiệu (Signal).
2. Đường ống (Pipe).
3. Vùng nhớ chia sẻ.
4. Trao đổi thông điệp (Message).
5. Socket.

ThS. GVC Tô Oai Hùng

51

51

### Tín Hiệu

- ❖ Sử dụng tín hiệu để thông báo cho tiến trình khi có một sự kiện xảy ra.
- ❖ Mỗi tiến trình có một bảng biểu diễn các tín hiệu khác nhau.
- ❖ Mỗi tín hiệu có một trình xử lý tương ứng.
- ❖ Các tín hiệu được gọi đi bởi:
  - Phần cứng (ví dụ lỗi do các phép tính số học).
  - Kernel gọi đến một tiến trình (ví dụ: báo cho tiến trình khi có một thiết bị I/O rồi).
  - Một tiến trình gọi đến một tiến trình khác (ví dụ: tiến trình cha yêu cầu một tiến trình con kết thúc).
  - Người dùng (ví dụ nhấn phím Ctrl-C để ngắt xử lý của tiến trình).

ThS. GVC Tô Oai Hùng

52

52

## Tín Hiệu

### ❖ Một số tín hiệu của UNIX:

| Tín hiệu | Mô tả                                              |
|----------|----------------------------------------------------|
| SIGINT   | Người dùng nhấn phím DEL để ngắt xử lý tiến trình  |
| SIGQUIT  | Yêu cầu thoát xử lý                                |
| SIGILL   | Tiến trình xử lý một chỉ thị bất hợp lệ            |
| SIGKILL  | Yêu cầu kết thúc một tiến trình                    |
| SIGFPT   | Lỗi floating – point xảy ra ( chia cho 0)          |
| SIGPIPE  | Tiến trình ghi dữ liệu vào pipe mà không có reader |
| SIGSEGV  | Tiến trình truy xuất đến một địa chỉ bất hợp lệ    |
| SIGCLD   | Tiến trình con kết thúc                            |
| SIGUSR1  | Tín hiệu 1 do người dùng định nghĩa                |
| SIGUSR2  | Tín hiệu 2 do người dùng định nghĩa                |

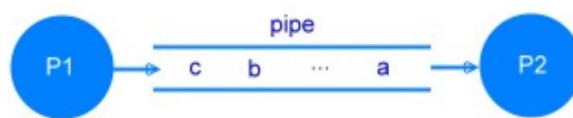
ThS. GVC Tô Oai Hùng

53

53

## Đường Ống

- ❖ Dữ liệu xuất của tiến trình này được chuyển đến làm dữ liệu nhập cho tiến trình kia dưới dạng một dòng các byte.
- ❖ Khi một đường ống (*pipe*) được thiết lập giữa hai tiến trình:
  - Một tiến trình ghi dữ liệu vào pipe.
  - Một tiến trình đọc dữ liệu từ pipe.
- ❖ Thứ tự dữ liệu truyền qua pipe: FIFO.
- ❖ Một pipe có kích thước giới hạn (thường là 4096 ký tự).



ThS. GVC Tô Oai Hùng

54

54

## Đường Ống

- ❖ Một tiến trình có thể sử dụng một pipe do nó tạo ra hay kế thừa từ tiến trình cha.
- ❖ Vai trò của hệ điều hành:
  - Cung cấp các lời gọi hệ thống (hàm) read/write cho các tiến trình thực hiện thao tác đọc/ghi dữ liệu trong pipe
  - Đồng bộ hóa việc truy xuất:
    - Nếu pipe trống: Tiến trình đọc bị khóa.
    - Nếu pipe đầy: Tiến trình ghi bị khóa.
- ❖ Pipe có nhiệm vụ nhận dữ liệu xuất từ một lệnh và đưa vào như dữ liệu nhập cho lệnh kế tiếp theo dạng sau:

ThS. GVC Tô Oai Hùng

55

55

## Đường Ống

`command1 | command2 | command3 ...`

(dấu xỏ đứng | chỉ định cho một pipe)

- ❖ Ví dụ trong cửa sổ Command Prompt của Windows:

`type | find`

(đường ống xuất từ lệnh `type` đi vào lệnh `find`):

```
C:\>TYPE t.txt | FIND "FTP"
Đây là tập tin test FTP
```

```
C:\>TYPE t.txt | FIND "aaa"
```

```
C:\>TYPE t.txt | FIND "ti"
Đây là tập tin test FTP
```

ThS. GVC Tô Oai Hùng

56

56

## Đường Ống

- ❖ Ví dụ trong cửa sổ Command Prompt của Linux:

`ls | more`

```
hung@hung-virtual-machine:~$ ls
Desktop Documents Downloads Music Pictures Public snap Templates Videos
hung@hung-virtual-machine:~$ ls | more
Desktop
Documents
Downloads
Music
Pictures
Public
snap
Templates
Videos
hung@hung-virtual-machine:~$
```

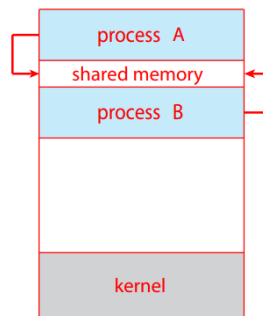
ThS. GVC Tô Oai Hùng

57

57

## Vùng Nhớ Chia Sẻ

- ❖ Cho phép nhiều tiến trình cùng truy xuất đến một vùng nhớ chung gọi là vùng nhớ chia sẻ (*shared memory*).
- ❖ Các tiến trình chia sẻ một vùng nhớ vật lý thông qua không gian địa chỉ của tiến trình.



ThS. GVC Tô Oai Hùng

Shared memory.

58

58

## Vùng Nhớ Chia Sẻ

- ❖ Vùng nhớ chia sẻ tồn tại độc lập với các tiến trình.
- ❖ Một tiến trình muốn truy xuất đến vùng nhớ chia sẻ, tiến trình phải gắn kết vùng nhớ chung đó vào không gian địa chỉ riêng của từng tiến trình để thao tác trên đó.

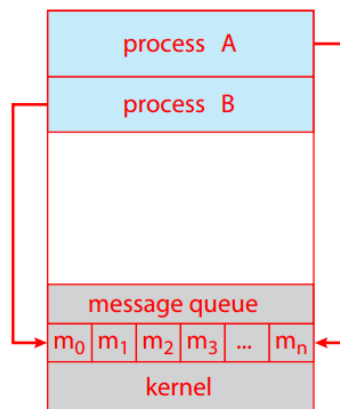
ThS. GVC Tô Oai Hùng

59

59

## Trao Đổi Thông Điệp

- ❖ Các tiến trình liên lạc với nhau thông qua việc gửi thông điệp (*Message*).



ThS. GVC Tô Oai Hùng

Message passing.

60

60

## Trao Đổi Thông Điệp

- ❖ HĐH cung cấp các hàm IPC chuẩn (*Interprocess communication*):
  - Send ([destination], message): Gửi một thông điệp đến *destination*.
  - Receive ([source], message): Nhận một thông điệp từ *source*.
- ❖ Khi hai tiến trình muốn liên lạc với nhau:
  - Thiết lập một mối liên kết giữa hai tiến trình.
  - Sử dụng các hàm IPC thích hợp để trao đổi thông điệp.
  - Hủy liên kết.

ThS. GVC Tô Oai Hùng

61

61

## Sockets

- ❖ Được dùng để trao đổi dữ liệu giữa các máy tính trên mạng
- ❖ Một socket là một thiết bị truyền thông hai chiều để gửi và nhận dữ liệu giữa các máy trên mạng.
- ❖ Mỗi socket là một đầu nối giữa một máy đến một máy tính khác.
- ❖ Thao tác đọc/ghi là sự trao đổi dữ liệu ứng dụng trên nhiều máy khác nhau.
- ❖ Sử dụng socket có thể mô phỏng hai phương thức liên lạc trong thực tế:
  - Liên lạc thư tín (socket đóng vai trò bưu cục).

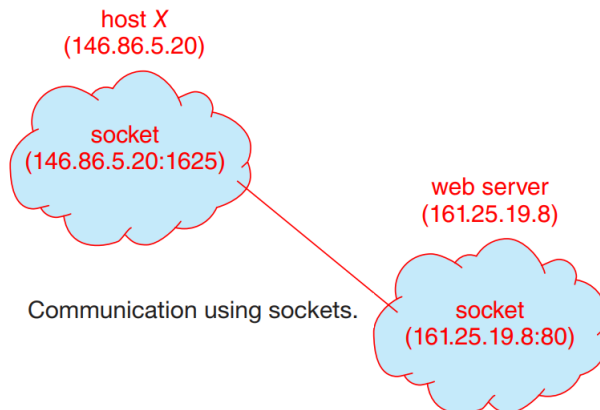
ThS. GVC Tô Oai Hùng

62

62

## Sockets

- Liên lạc điện thoại (socket đóng vai trò tổng đài).



ThS. GVC Tô Oai Hùng

63

63

## Sockets

- ❖ Các bước trao đổi dữ liệu:
  - Tạo socket.
  - Gắn kết socket với một địa chỉ (IP:Port).
  - Liên lạc:
    - Liên lạc trong chế độ không kết nối.
    - Liên lạc trong chế độ có kết nối.
  - Hủy socket.
- ❖ Trong hình trên, kết nối giữa Client với Server gồm một cặp socket: (146.86.5.20:1625) của máy Client (host X) với (161.25.19.8:80) của Web server.

ThS. GVC Tô Oai Hùng

64

64



### 3.4 Đồng Bộ Hóa Tiến Trình

1. Vấn đề tranh chấp tài nguyên.
2. Các giải pháp đồng bộ hoạt động của các tiến trình.

ThS. GVC Tô Oai Hùng

65

65

#### 3.4.1 Vấn Đề Tranh Chấp Tài Nguyên

- ❖ Trong một hệ thống có nhiều tiến trình cùng chạy.
- ❖ Các tiến trình có thể chia sẻ tài nguyên chung (file system, CPU...).
- ❖ Nhiều tiến trình truy xuất đồng thời một tài nguyên mang bản chất không chia sẻ được  
→ Xảy ra vấn đề *Tranh đoạt điều khiển (Race Condition)*.

ThS. GVC Tô Oai Hùng

66

66

## Race Condition

- ❖ Ví dụ 1: Đếm số người truy cập Website, có 2 tiến trình chia sẻ đoạn code cập nhật biến đếm counter như sau:

counter = 0

 P1  
 n = counter;  
 n = n + 1;  
 counter = n;

 P2  
 n = counter;  
 n = n + 1;  
 counter = n;

counter?

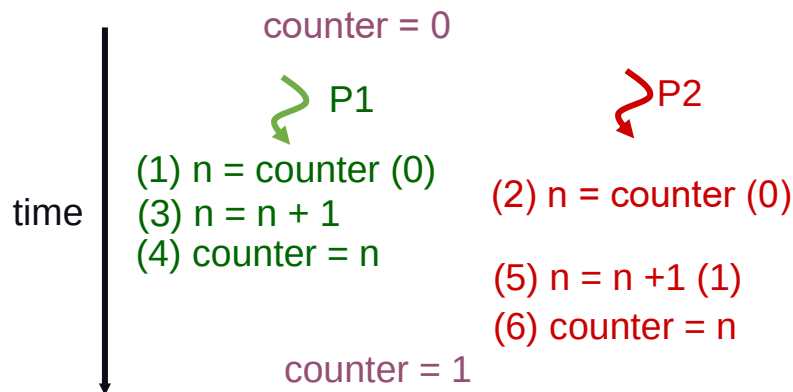
ThS. GVC Tô Oai Hùng

67

67

## Race Condition

- ❖ Đếm số người truy cập Website: Tình huống 1.



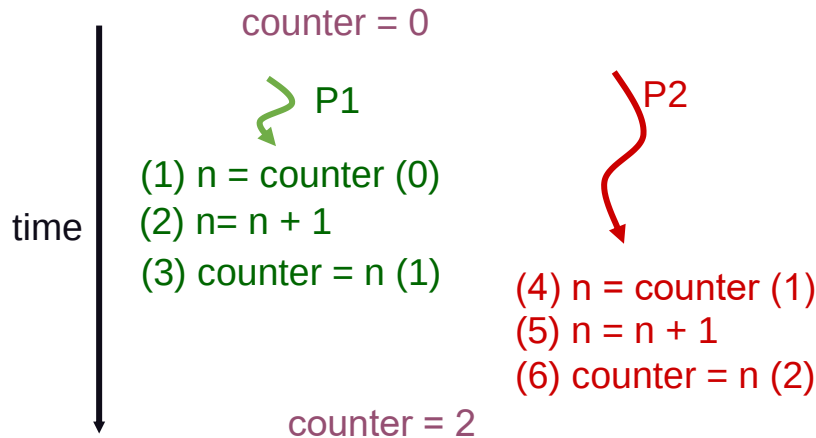
ThS. GVC Tô Oai Hùng

68

68

## Race Condition

### ❖ Đếm số người truy cập Website: Tình huống 2.



ThS. GVC Tô Oai Hùng

69

69

## Race Condition

### ❖ Ví dụ 2: Bài toán rút tiền từ ngân hàng.

```
if(taikhoan - tienrut >= 0)
 taikhoan = taikhoan - tienrut;
else
 error("Khong the rut tien!");
```

### ❖ Giả sử trong tài khoản hiện còn 800, P1 muốn rút 500 và P2 muốn rút 400.

### ❖ Nếu xảy ra tình huống như sau:

- P1 kiểm tra điều kiện đúng ( $\text{if}(800 - 500 \geq 0)$ ) và hết thời gian xử lý, hệ điều hành cấp CPU cho P2.
- P2 cũng kiểm tra điều kiện đúng ( $\text{if}(800 - 400 \geq 0)$ ), do P1 chưa rút tiền) và rút 400.

ThS. GVC Tô Oai Hùng

70

70

## Race Condition

- Giá trị của `taikhoan` được cập nhật lại là 400.
- Khi P1 được tái kích hoạt, nó sẽ không kiểm tra lại điều kiện (`taikhoan - tienrut >= 0`), vì đã kiểm tra rồi, mà thực hiện rút tiền.
- Giá trị của `taikhoan` sẽ lại được cập nhật thành -100.
- Tình huống lỗi xảy ra!

ThS. GVC Tô Oai Hùng

71

71

## Race Condition

- ❖ Lý do xảy ra race condition:
  - Hai hoặc nhiều tiến trình cùng đọc/ghi dữ liệu trên một vùng nhớ chung.
  - Một tiến trình xen vào quá trình truy xuất tài nguyên của một tiến trình khác.
- ❖ Giải pháp:
  - Đảm bảo cho một tiến trình hoàn tất việc truy xuất tài nguyên chung trước khi có tiến trình khác can thiệp.

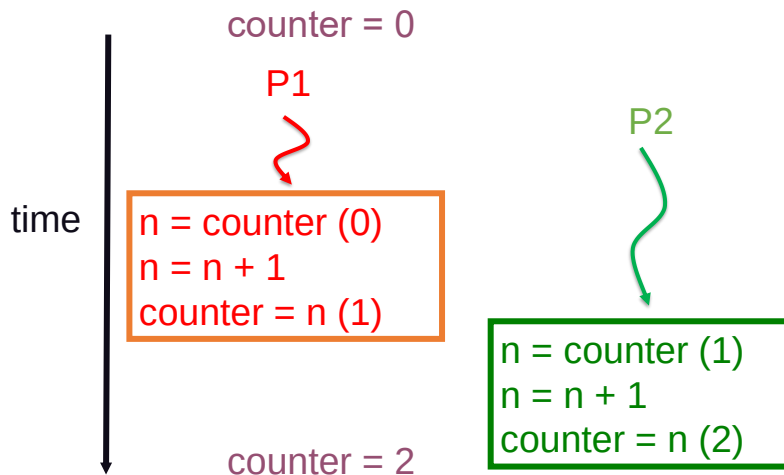
ThS. GVC Tô Oai Hùng

72

72

## Race Condition

- ❖ Giải pháp cho ví dụ 1 (tương tự cho ví dụ 2):



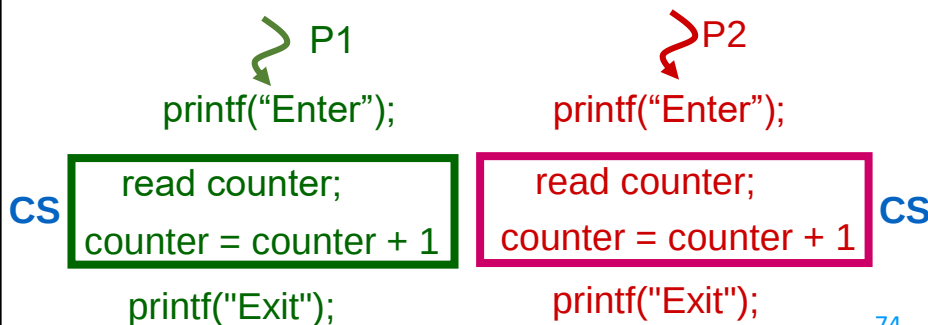
ThS. GVC Tô Oai Hùng

73

73

## Miền Găng & Khả Năng Độc Quyền

- ❖ Miền găng (*Critical Section* - CS) là đoạn chương trình có khả năng gây ra tình trạng race condition.
- ❖ Để không xảy ra tình trạng race condition → bảo đảm tính "độc quyền truy xuất" (*Mutual Exclusion*) cho miền găng (CS).



74

74

## Giải Pháp Cho Vấn Đề Tranh Chấp Tài Nguyên

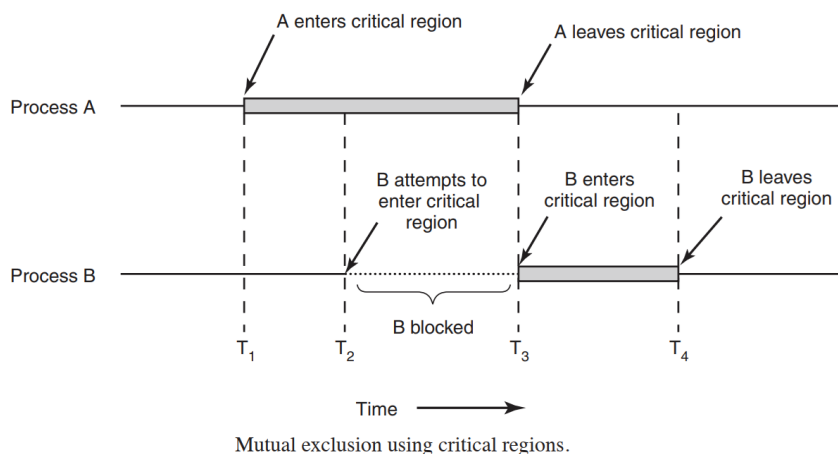
- ❖ Đồng bộ hóa tiến trình: bảo đảm tại một thời điểm chỉ có duy nhất một tiến trình được xử lý trong miền găng.
- ❖ Bốn mục tiêu đồng bộ hóa tiến trình:
  1. Mutual exclusion (độc quyền): Không có hai tiến trình cùng ở trong miền găng cùng lúc.
  2. Không có giả định nào đặt ra về tốc độ của các tiến trình hoặc số lượng CPU.
  3. Progress: Một tiến trình bên ngoài miền găng không được ngăn cản các tiến trình khác vào miền găng.
  4. Bounded waiting: Không có tiến trình nào phải chờ vô hạn để được vào miền găng.

ThS. GVC Tô Oai Hùng

75

75

## Giải Pháp Cho Vấn Đề Tranh Chấp Tài Nguyên



ThS. GVC Tô Oai Hùng

76

76

## Giải Pháp Cho Vấn Đề Tranh Chấp Tài Nguyên

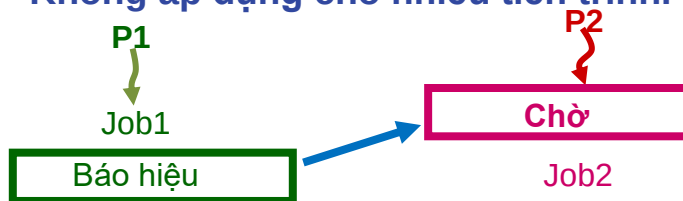
### ❖ Mô hình đảm bảo Mutual Exclusion:

- Thêm các đoạn code đồng bộ hóa vào chương trình gốc:



### ❖ Mô hình tổ chức phối hợp giữa 2 tiến trình:

- Thêm các đoạn code đồng bộ hóa vào 2 chương trình gốc.
- Không áp dụng cho nhiều tiến trình.



77

77

## 3.4.2 Các Giải Pháp Đồng Bộ Hoạt Động của Các Tiến Trình

### ❖ Nhóm giải pháp Busy Waiting.

- Phần mềm:
  - Sử dụng các biến cờ hiệu.
  - Sử dụng việc kiểm tra luân phiên.
  - Giải pháp của Peterson.
- Phần cứng:
  - Cấm ngắt.
  - Chỉ thị TSL.

### ❖ Nhóm giải pháp Sleep & Wakeup.

- Mutex.
- Semaphore.
- Monitor.
- Message.

ThS. GVC Tô Oai Hùng

78

78

## Các Giải Pháp "Busy waiting"

**While (chưa có quyền) donothing() ;**

CS;

**Từ bỏ quyền sử dụng CS**

### ❖ Nhận xét:

- Không đòi hỏi sự trợ giúp của hệ điều hành.
- Tiếp tục tiêu thụ CPU trong khi chờ đợi vào miền găng.

ThS. GVC Tô Oai Hùng

79

79

## Sử Dụng Biến Cờ Hiệu

### ❖ Sử dụng biến lock làm khóa chốt, khởi động là 0.

- lock = 0: CS rảnh.
- lock = 1: CS bận (có tiến trình đang truy cập).

### ❖ Tiến trình muốn vào miền găng phải kiểm tra giá trị của lock.

- lock = 0:
  - Đặt lock = 1
  - Vào miền găng.
  - Sau khi ra khỏi miền găng đặt lock = 0.
- lock = 1: Chờ.

```
while (TRUE) {
 while (lock == 1); // wait
 lock = 1;
 critical-section ();
 lock = 0;
 Noncritical-section ();
}
```

ThS. GVC Tô Oai Hùng

80

80



## Sử Dụng Biến Cờ Hiệu

### ❖ Nhận xét:

- Có thể áp dụng cho nhiều tiến trình.
- Hai tiến trình có thể cùng ở miền găng tại một thời điểm:

**P1**

NonCS

```
while (lock == 1); // wait
lock = 1;
```

CS:

```
lock = 0;
```

NonCS;

```
int lock = 0
```

**P2**

NonCS

```
while (lock == 1); // wait
lock = 1;
```

CS:

```
lock = 0;
```

NonCS;

81

81

## Sử Dụng Biến Cờ Hiệu

- Lúc đầu lock = 0.
- P1 và P2 cùng thực thi while(lock == 1);
- P1 gán lock = 1 và vào CS.
- P2 gán lock = 1 và vào CS.
- Cả hai P1 và P2 cùng vào CS.

82

## Kiểm Tra Luân Phiên

- ❖ Hai tiến trình sử dụng chung biến turn, khởi động = 0 hoặc 1.
  - turn = 0: P0 được vào miền găng, P1 chờ.
  - turn = 1: P1 được vào miền găng, P0 chờ.
  - Khi tiến trình P0 rời khỏi miền găng: Đặt turn = 1.
  - Khi tiến trình P1 rời khỏi miền găng: Đặt turn = 0.

```
while (TRUE) P0
{
 while (turn != 0); // wait
 critical-section ();
 turn = 1;
 Noncritical-section ();
}
Th
```

```
while (TRUE) P1
{
 while (turn != 1); // wait
 critical-section ();
 turn = 0;
 Noncritical-section ();
}
```

83

83

## Kiểm Tra Luân Phiên

- ❖ Nhận xét:
  - Chỉ áp dụng cho 2 tiến trình.
  - Bảo đảm Mutual Exclusion: Ngăn chặn được tình trạng hai tiến trình cùng vào miền găng do kiểm tra biến turn.
  - Vi phạm Progress: Một tiến trình ở ngoài miền găng có thể ngăn chặn tiến trình khác vào miền găng.

ThS. GVC Tô Oai Hùng

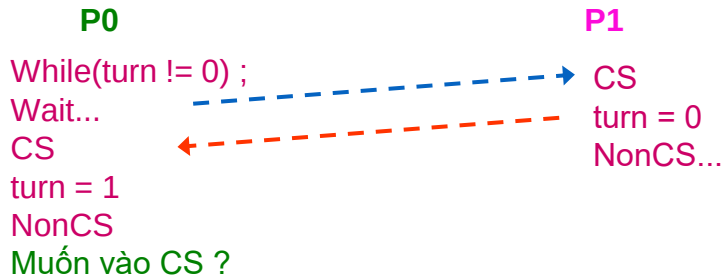
84

84

## Kiểm Tra Luân Phiên

### ❖ Vi phạm Progress:

int turn = 1



**P0 không vào được CS lần 2 khi P1 dừng trong NonCS !**

ThS. GVC Tô Oai Hùng

85

85

## Peterson

### ❖ Kết hợp ý tưởng của 1 & 2, các tiến trình chia sẻ các biến:

- int turn // đến phiên tiến trình nào
- Int interested [2] // tiến trình muốn vào CS  
interested [i] = TRUE: P<sub>i</sub> vào miền găng.

### ❖ Khởi đầu:

- interested [0] = interested [1] = FALSE;
- turn = 0 hay 1.

### ❖ P<sub>i</sub> muốn vào được CS:

- interested [ j ] = FALSE // j = 1 - i
- turn = i

### ❖ P<sub>i</sub> rời khỏi miền găng:

- interested [i] = FALSE

ThS. GVC Tô Oai Hùng

86

86

## Peterson

```
#define FALSE 0
#define TRUE 1
#define N 2 /*number of processes */
int turn; /* whose turn is it? */
int interested [N]; /* all values initially 0 (FALSE) */
void enter_region (int process) { /* process is 0 or 1 */
 int other = 1 - process; /* the opposite of process */
 interested [process] = TRUE; /* show that you are interested */
 turn = process; /* set flag */
 while (turn == process && interested[other] == TRUE) ;
}
void leave_region (int process) { /* process: who is leaving */
 interested [process] = FALSE;
}
```

67

87

## Peterson

- ❖ **Nhận xét: Đáp ứng được cả 3 điều kiện.**
  - **Mutual Exclusion:**
    - P<sub>i</sub> chỉ có thể vào CS khi: interested[j] == False, turn == i.
    - Do biến turn chỉ có thể nhận giá trị 0 hay 1 nên chỉ có 1 tiến trình vào CS.
  - **Progress:**
    - Sử dụng 2 biến interested[i] riêng biệt => tiến trình đối phương không khoá lẫn nhau được.
  - **Bounded Wait :** interested [i] và turn đều có thay đổi giá trị.
  - Không thể mở rộng cho N tiến trình.

ThS. GVC Tô Oai Hùng

88

88

## Cấm Ngắt

- ❖ Cho phép tiến trình cấm tất cả các ngắt (kể cả ngắt đồng hồ) trước khi vào CS, và phục hồi ngắt khi ra khỏi CS.
- ❖ → Hệ thống không thể tạm dừng hoạt động của tiến trình đang xử lý để cấp phát CPU cho tiến trình khác.
- ❖ Nhận xét:
  - Thiếu thận trọng: Cho phép tiến trình người dùng được phép thực hiện lệnh cấm ngắt.
  - Hệ thống có nhiều bộ xử lý: Lệnh cấm ngắt chỉ có tác dụng trên bộ xử lý đang

ThS. GVC Tô Oai Hùng

89

89

## Cấm Ngắt

xử lý tiến trình, còn các tiến trình hoạt động trên các bộ xử lý khác vẫn có thể vào CS → không đảm bảo Mutual Exclusion.

NonCS;

Disable Interrupt;

CS;

Enable Interrupt;

NonCS;

ThS. GVC Tô Oai Hùng

90

90

## Chỉ Thị TSL (Test-and-Set)

- ❖ Giải pháp này yêu cầu sự trợ giúp của cơ chế phần cứng.
- ❖ Hệ thống cung cấp một lệnh đặc biệt cho phép kiểm tra và cập nhật nội dung một vùng nhớ chung.
- ❖ Hai chỉ thị TSL xử lý đồng thời (trên hai bộ xử lý khác nhau) sẽ được xử lý tuần tự.
- ❖ Đảm bảo truy xuất độc quyền:
  - Sử dụng thêm một biến lock.
  - Khởi gán lock = FALSE.
  - Tiến trình muốn vào CS phải kiểm tra nếu lock = FALSE → vào CS.

ThS. GVC Tô Oai Hùng

91

91

## Chỉ Thị TSL

- ❖ Chỉ thị TSL:

```
boolean test_and_set (boolean *target) {
 boolean rv = *target;
 *target = true;
 return rv;
}
```

- ❖ Tiến trình phải kiểm tra giá trị của biến lock trước khi vào miền găng:

```
do {
 while (test_and_set (&lock));
 Critical-section();
 lock = false;
 Noncritical-section();
} while (true);
```

92

92

## Chỉ Thị TSL

- ❖ Nhận xét về TSL:
  - Cần được sự hỗ trợ của cơ chế phần cứng.
  - Không dễ cài đặt, nhất là trên các máy có nhiều bộ xử lý.
  - Dễ mở rộng cho N tiến trình.
- ❖ Nhận xét chung về các giải pháp Busy waiting:
  - Sử dụng CPU không hiệu quả do liên tục kiểm tra điều kiện khi chờ vào CS.
  - Khắc phục: Khoá các tiến trình chưa đủ điều kiện vào CS, nhường CPU cho tiến trình khác → giải pháp Sleep & Wake up.

ThS. GVC Tô Oai Hùng

93

93

## Các Giải Pháp "Sleep & Wake up"

- ❖ Các tiến trình phải từ bỏ CPU khi chưa được vào CS.
- ❖ Khi CS trống, sẽ được đánh thức để vào CS.
- ❖ Cần phải sử dụng các thủ tục do hệ điều hành cung cấp để thay đổi trạng thái tiến trình.

```
if (chưa có quyền) Sleep();
```



```
CS;
```

```
Wakeup(somebody);
```



ThS. GVC Tô Oai Hùng

94

94

## Sleep & Wake up

- ❖ Hệ Điều hành hỗ trợ 2 hàm đặc quyền:
  - Sleep(): Tiến trình tự gọi hàm này để chuyển sang trạng thái Blocked.
  - WakeUp (P): Tiến trình P nhận trạng thái Ready.
- ❖ Khi một tiến trình chưa đủ điều kiện vào CS → gọi Sleep () → chuyển sang trạng thái bị khóa cho đến khi có một tiến trình khác gọi WakeUp để giải phóng cho nó.
- ❖ Một tiến trình khi ra khỏi CS sẽ gọi WakeUp (P) để đánh thức một tiến trình khác (P) đang bị khóa, để tiến trình này vào CS.

ThS. GVC Tô Oai Hùng

95

95

## Sleep & Wake up

```

int busy; //busy = 0: CS trống
int blocked; //số tiến trình bị Blocked chờ vào CS
while (TRUE) {
 if (busy){
 blocked = blocked + 1;
 sleep();
 }
 else //busy =0
 busy = 1;
 Critical-section();
 busy = 0;
 if(blocked){
 wakeup(process);
 blocked = blocked - 1;
 }
 Noncritical-section();
}

```

ThS. GVC Tô Oai Hùng

96

96



## Sleep & Wake up

- ❖ Có thể xảy ra mâu thuẫn truy xuất:
  - Giả sử P1 vào CS, khi chưa xử lý xong P1 hết thời gian sử dụng CPU, P2 được kích hoạt.
  - P2 muốn vào CS nhưng kiểm tra thấy P1 đang ở trong CS → P2 tăng giá trị biến blocked và sẽ gọi Sleep để tự khóa.
  - Trước khi P2 gọi Sleep, P1 lại được tái kích hoạt, xử lý xong và ra khỏi CS.
  - P1 thấy có một tiến trình đang chờ (blocked=1) nên gọi WakeUp và giảm giá trị của blocked (blocked = 0).
  - Do P2 chưa thực hiện Sleep nên không thể nhận tín hiệu WakeUp.

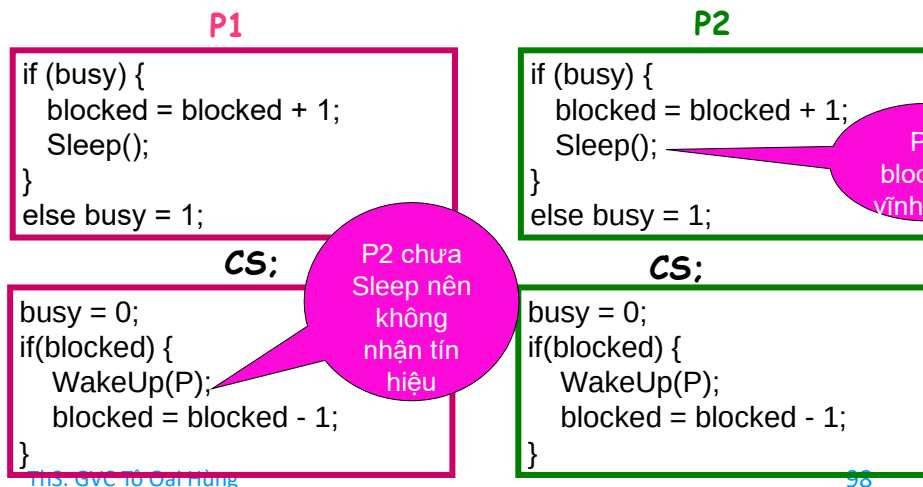
ThS. GVC Tô Oai Hùng

97

97

## Vấn Đề với Sleep & Wake up

- Khi P2 được tiếp tục xử lý, nó mới gọi Sleep và tự khóa vĩnh viễn.



ThS. GVC Tô Oai Hùng

98

98

## Mutex Locks

- ❖ Là giải pháp đơn giản được sử dụng để ngăn chặn race condition.
- ❖ Mutex là một biến có hai giá trị: khóa/không khóa.
- ❖ Hai hàm đặc quyền được hỗ trợ từ phần cứng:
  - mutex-lock: tiến trình muốn vào CS.
  - mutex-unlock: tiến trình ra khỏi CS, mở khóa cho các tiến trình khác có thể vào CS.
- ❖ Một tiến trình muốn vào CS, gọi mutex-lock và trước khi rời khỏi CS gọi mutex-unlock.

ThS. GVC Tô Oai Hùng

99

99

## Mutex Locks

```
mutex_lock() {
 while (!available); /* busy wait */
 available = false;
}
mutex_unlock() {
 available = true;
}
```

```
do {
 mutex_lock ();
 Critical_section
 mutex_unlock ();
 Noncritical_section
} while (true);
```

ThS. GVC Tô Oai Hùng

100

100

## Semaphore

- ❖ Được Dijkstra đề xuất vào 1965.
- ❖ Một semaphore  $s$  là cấu trúc dữ liệu gồm có:
  - Một biến nguyên dương count:
    - + Khi  $s.count \geq 0$ : Thì count chính là số process có thể thực thi wait(s) mà không bị blocked
    - + Khi  $s.count < 0$ : Thì  $|count|$  chính là số process đang đợi trên hàng đợi semaphore  $s$ .
  - Hàng đợi queue: Danh sách các tiến trình đang bị khóa trên semaphore  $s$ .
  - Hai phương thức:

ThS. GVC Tô Oai Hùng

101

101

## Semaphore

- wait(semaphore  $s$ ):
  - + Khi Wait() được gọi, giá trị count của semaphore  $s$  bị giảm đi 1.
  - + Nếu count âm, tiến trình này sẽ bị đưa vào hàng đợi semaphore  $s$  và bị khóa (block) lại.
- signal(semaphore  $s$ ):
  - + Khi phương thức signal() được gọi, giá trị count của semaphore  $s$  tăng lên 1.
  - + Nếu giá trị này nhỏ hơn hoặc bằng 0, một tiến trình sẽ được chọn trong danh sách hàng đợi semaphore (theo FIFO) và bị đánh thức dậy (wakeup) để đưa vào hàng đợi ready và chờ nhận CPU để thực hiện.

ThS. GVC Tô Oai Hùng

102

102

## Cài Đặt Semaphore (Sleep & Wakeup)

```
struct semaphore {
 int count;
 queueType queue;
};
```

```
void wait (semaphore s) {
 s.count--;
 if (s.count < 0) {
 /* place this process in s.queue */;
 /* block this process */;
 }
}
```

ThS. GVC Tô Oai Hùng

103

103

## Cài Đặt Semaphore (Sleep & Wakeup)

```
void signal (semaphore s) {
 s.count++;
 if (s.count <= 0) {
 /* remove a process P from s.queue */;
 /* place process P on ready list - wakeup(P)*/;
 }
}
```

- ❖ Việc hiện thực Semaphore phải đảm bảo tính chất nguyên tử và độc quyền:
  - Tức không được xảy ra trường hợp 2 process cùng đang ở trong thân wait(s) và signal(s) (cùng semaphore s) tại một thời điểm.
  - Ngay cả với hệ thống multiprocessor.

ThS. GVC Tô Oai Hùng

104

104

## Sử Dụng Semaphore

- ❖ Truy xuất độc quyền với semaphore:
  - Có hai trường hợp: Nếu chỉ duy nhất một tiến trình được vào CS, khởi tạo  $s.count = 1$ . Nếu cho phép  $k$  tiến trình vào CS, khởi tạo  $s.count = k$ .
  - Cho phép bảo đảm nhiều tiến trình cùng truy xuất đến CS mà không có sự mâu thuẫn truy xuất.
  - $N$  tiến trình cùng cấu trúc chương trình sau:

```
while(TRUE) {
 wait(s);
 Critical-section();
 signal(s);
 Noncritical-section();
}
```

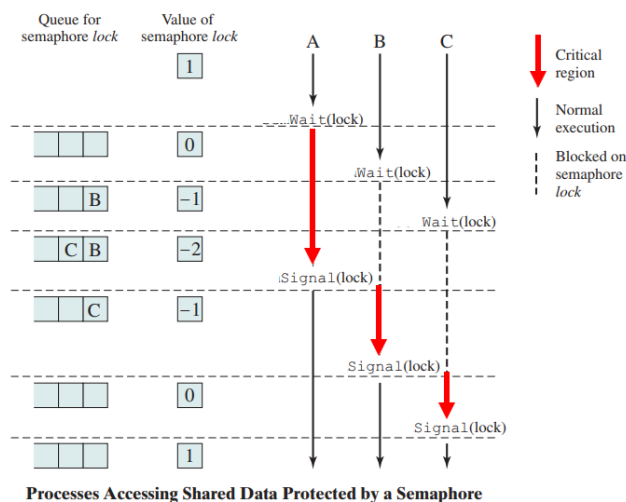
ThS. GVC Tô Oai Hùng

105

105

## Sử Dụng Semaphore

- ❖ Ví dụ truy xuất độc quyền với semaphore:



106

106

## Sử Dụng Semaphore

❖ Giải thích hình trên bằng mã lệnh:

```
void wait (semaphore s) {
 s.count--;
 if (s.count < 0) {
 /* place this process in s.queue */;
 /* block this process */;
 }
}
```

```
void signal (semaphore s) {
 s.count++;
 if (s.count <= 0) {
 /* remove P from s.queue */;
 /* wakeup(P)*/;
 }
}
```

// Process A

```
while(TRUE)
{
 wait(s);
 Critical-section();
 signal(s);
 Noncritical-section();
}
```

// Process B

```
while(TRUE)
{
 wait(s);
 Critical-section();
 signal(s);
 Noncritical-section();
}
```

// Process C

```
while(TRUE)
{
 wait(s);
 Critical-section();
 signal(s);
 Noncritical-section();
}
```

ThS. GVC Tô Oai Hùng

107

107

## Sử Dụng Semaphore

❖ Tổ chức đồng bộ hóa với Semaphore:

- Một tiến trình phải đợi một tiến trình khác hoàn tất thao tác nào đó mới có thể bắt đầu hay tiếp tục xử lý.
- Hai tiến trình chia sẻ một semaphore s, khởi tạo count = 0.

```
P1:
while (TRUE)
{
 job1();
 signal (s); // đánh thức P2
}
```

```
P2:
while (TRUE)
{
 wait (s); // chờ P1
 job2();
}
```

- Hàm job2() chỉ được thực thi sau khi lệnh hàm job1() được thực thi.

ThS. GVC Tô Oai Hùng

108

108

## Nhận Xét Semaphore

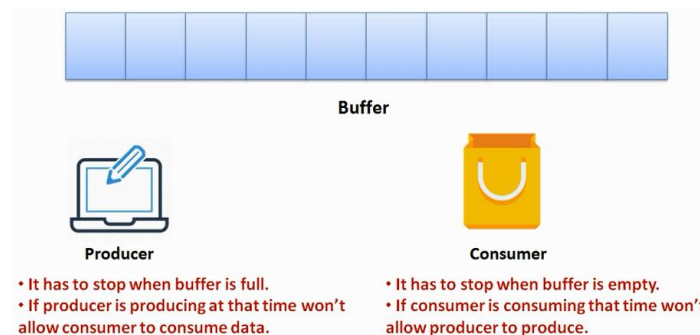
- ❖ Semaphore được xem như là một tài nguyên:
  - Các tiến trình "yêu cầu" semaphore s: Gọi wait (s).
  - Nếu không hoàn tất được wait (s): Chưa được cấp tài nguyên → blocked, được đưa vào s.queue.
- ❖ Cần có sự hỗ trợ của HĐH: Sleep() & Wakeup().
- ❖ Là một cơ chế tốt để thực hiện đồng bộ.
- ❖ Mở rộng cho N tiến trình.
- ❖ Khó sử dụng đúng: Nếu thiếu hoặc đặt sai vị trí wait() và signal().

ThS. GVC Tô Oai Hùng

109

109

## The Producer-Consumer Problem



- ❖ **Producer:**
  - Sản xuất một sản phẩm và đặt vào buffer.
  - Buffer giới hạn chỉ chứa được n sản phẩm.

ThS. GVC Tô Oai Hùng

110

110

## The Producer-Consumer Problem

- Khi buffer đã chứa n sản phẩm, producer không thể đưa tiếp sản phẩm vào buffer mà phải chờ đến khi buffer có chỗ trống.
- ❖ Consumer:
  - Tiêu thụ mỗi lần một sản phẩm.
  - Sản phẩm được lấy ra từ buffer.
  - Khi buffer rỗng, consumer không thể lấy sản phẩm để tiêu thụ mà phải chờ đến khi có ít nhất 1 sản phẩm trong buffer.

ThS. GVC Tô Oai Hùng

111

111

## The Producer-Consumer Problem

### 1. Sử dụng giải pháp sleep & wakeup:

```
#define N 100 /* number of slots in the buffer */
int count = 0; /* number of items in the buffer */
void producer(void)
{
 int item;
 while (TRUE) { /* repeat forever */
 item = produce_item(); /* generate next item */
 if (count == N) sleep(); /* if buffer is full, go to sleep */
 insert_item(item); /* put item in buffer */
 count = count + 1; /* increment count of items in buffer */
 if (count == 1) wakeup(consumer); /* was buffer empty? */
 }
}
```

ThS. GVC Tô Oai Hùng

112

112



## The Producer-Consumer Problem

```
void consumer(void)
{
 int item;
 while (TRUE) {
 if (count == 0) sleep(); /* repeat forever */
 item = remove_item(); /* if buffer is empty, got to sleep */
 count = count - 1; /* take item out of buffer */
 if (count == N - 1) wakeup(producer); /* decrement count of items in buffer */
 consume_item(item); /* was buffer full? */
 }
}
```

ThS. GVC Tô Oai Hùng

113

113

## The Producer-Consumer Problem

### ❖ Vấn đề:

- Ban đầu count = 0.
- C (consumer) thấy count = 0 → chuẩn bị sleep, nhưng bị block.
- P (producer): count = count + 1, gọi wakeup để đánh thức C.
- Tuy nhiên, có thể C vẫn chưa sleep một cách hợp lý, vì vậy tín hiệu wakeup bị mất.
- Khi C tiếp tục chạy → kiểm tra giá trị biến count đã đọc trước đó: count = 0 → sleep.
- Khi P lấp đầy bộ đệm (count = N) → sleep.
- Cả P và C đều sleep.

ThS. GVC Tô Oai Hùng

114

114

## The Producer-Consumer Problem

### 2. Sử dụng semaphore:

```
#define N 100 /* number of slots in the buffer */
typedef int semaphore; /* semaphores are a special kind of int */
semaphore mutex = 1; /* controls access to critical region */
semaphore empty = N; /* counts empty buffer slots */
semaphore full = 0; /* counts full buffer slots */
void producer(void) {
 int item;
 while (TRUE) { /* TRUE is the constant 1 */
 item = produce item(); /* generate something to put in buffer */
 wait(empty); /* decrement empty count */
 wait(mutex); /* enter critical region */
 insert_item(item); /* put new item in buffer */
 signal(mutex); /* leave critical region */
 signal(full); /* increment count of full slots */
 }
}
```

115

115

## The Producer-Consumer Problem

```
void consumer(void) {
 int item;
 while (TRUE) { /* infinite loop */
 wait(full); /* decrement full count */
 wait(mutex); /* enter critical region */
 item = remove item(); /* take item from buffer */
 signal(mutex); /* leave critical region */
 signal(empty); /* increment count of empty slots */
 consume_item(item); /* do something with the item */
 }
}
```

ThS. GVC Tô Oai Hùng

116

116

## Monitor

- ❖ Đề xuất bởi Hoare (1974) & Brinch (1975).
- ❖ Là giải pháp đồng bộ hoá do NNLT cung cấp.
  - Hỗ trợ cùng các chức năng như semaphore.
  - Dễ sử dụng và kiểm soát hơn semaphore.
    - Bảo đảm mutual exclusion một cách tự động.
    - Sử dụng biến điều kiện để thực hiện đồng bộ hóa.
- ❖ Là một module phần mềm, bao gồm:
  - Một hoặc nhiều thủ tục (procedure).
  - Một đoạn code khởi tạo (*initialization code*).
  - Các biến dữ liệu cục bộ (*local data variable*).

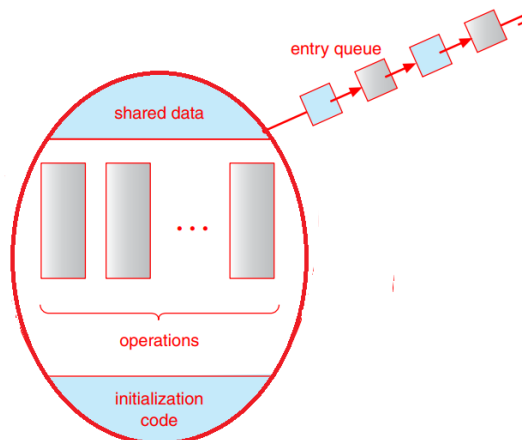
ThS. GVC Tô Oai Hùng

117

117

## Monitor

- ❖ Mô hình của một monitor đơn giản:



Schematic view of a monitor.

ThS. GVC Tô Oai Hùng

118

118

## Monitor

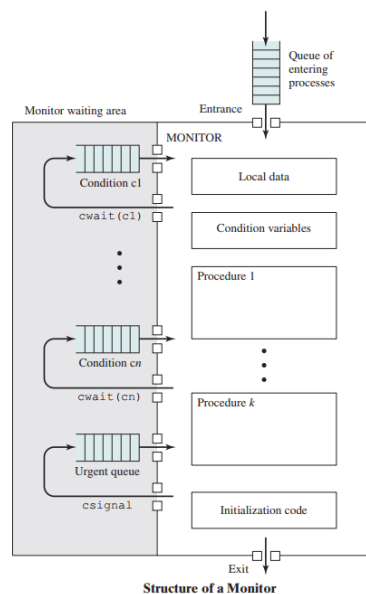
- ❖ Các đặc điểm chính của monitor:
  - Các biến dữ liệu cục bộ chỉ có thể truy cập được bằng thủ tục bên trong monitor (*encapsulation*).
  - Một tiến trình "vào monitor" bằng cách gọi một trong các phương thức trong monitor.
  - Tại một thời điểm, chỉ có một tiến trình duy nhất được hoạt động bên trong một monitor (*mutual exclusive*).
  - Các tiến trình không thể vào monitor → đưa vào hàng đợi.

ThS. GVC Tô Oai Hùng

119

119

## Cấu Trúc của Monitor



120

120

## Cấu Trúc của Monitor

- ❖ Chỉ có một tiến trình có thể vào monitor tại một thời điểm.
- ❖ Các tiến trình khác muốn vào monitor sẽ phải vào hàng đợi.
- ❖ Khi một tiến trình ở trong monitor, nó có thể tạm thời chặn (blocked) chính nó với điều kiện  $c$  bằng cách gọi `wait(c)`.
- ❖ Khi này, nó được đặt trong một hàng đợi của các tiến trình trên biến điều kiện  $c$  để chờ vào lại monitor khi điều kiện thay đổi và tiếp tục thực thi tại điểm trong chương trình sau lệnh gọi `wait(c)`.
- ❖ Nếu một tiến trình đang thực thi trong

ThS. GVC Tô Oai Hùng

121

121

## Cấu Trúc của Monitor

monitor phát hiện có sự thay đổi biến điều kiện  $c$ , nó gọi `signal(c)`.

- ❖ Khi này, nếu có một tiến trình đang bị khóa trong hàng đợi trên biến điều kiện  $c$ , nó được tái kích hoạt.

ThS. GVC Tô Oai Hùng

122

122

## Biến Điều Kiện

- ❖ Nhằm cho phép một tiến trình đợi "trong monitor", phải khai báo biến điều kiện (*condition variable*):

```
condition x, y;
```

- ❖ Các biến điều kiện đều cục bộ và chỉ được truy cập bên trong monitor.
- ❖ Đồng bộ hóa với các biến điều kiện:
  - `wait(x)`: Tiến trình gọi hàm sẽ bị blocked và đặt tiến trình này vào hàng đợi trên biến điều kiện x.
  - `signal(x)`: Giải phóng một tiến trình đang bị blocked trên biến điều kiện x.
  - + Nếu có nhiều tiến trình: Chỉ chọn một.

123

123

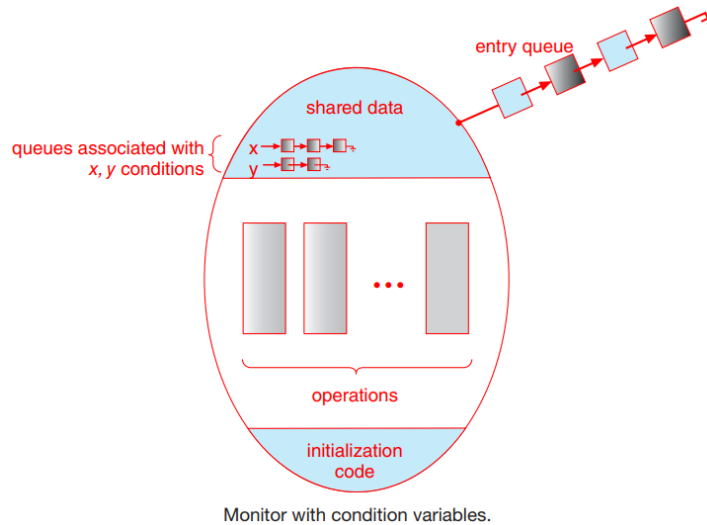
## Biến Điều Kiện

- + Nếu không có tiến trình: Không có tác dụng.
- ❖ Tiến trình sau khi gọi `signal(x)`:
  - Blocked, nhường quyền vào monitor cho tiến trình được đánh thức.
  - Tiếp tục xử lý hết chu kỳ, rồi blocked.

124

124

## Biến Điều Kiện



ThS. GVC Tô Oai Hùng

125

125

## Biến Điều Kiện

- ❖ Các tiến trình có thể đợi ở entry queue hoặc đợi ở các condition queue (x, y, ...).
- ❖ Khi thực hiện lời gọi wait(x), tiến trình sẽ được chuyển vào condition queue x.
- ❖ Lời gọi signal(x) chuyển một tiến trình từ condition queue x vào monitor.
- ❖ Khi đó, để bảo đảm mutual exclusion, tiến trình gọi signal(x) sẽ bị blocked và được đưa vào urgent queue.

ThS. GVC Tô Oai Hùng

126

126

## Cài Đặt Monitor

```

monitor monitor name
{
 /* shared variable declarations */
 function P1 (. . .) {
 . . .
 }
 function P2 (. . .) {
 . . .
 }
 . . .
 function Pn (. . .) {
 . . .
 }
 initialization code (. . .) {
 . . .
 }
}

```

ThS. GVC Tô Oai Hùng

127

127

## Cài Đặt Monitor

- ❖ Trình biên dịch chịu trách nhiệm thực hiện việc truy xuất đọc quyền dữ liệu trong monitor.
- ❖ Mỗi monitor có một hàng đợi toàn cục lưu các tiến trình đang chờ được vào monitor. Ngoài ra, mỗi biến điều kiện  $c$  cũng gắn với một hàng đợi  $f(c)$ .
- ❖ Hai thao tác trên biến điều kiện  $c$  được định nghĩa như sau:

```

wait(c)
{
 status(P) = blocked;
 enter(P , $f(c)$);
}

```

ThS. GVC Tô Oai Hùng

128

128



## Cài Đặt Monitor

```

signal(c)
{
 if (f(c) != NULL)
 {
 exit(Q, f(c)); // Q là tiến trình chờ
 // trên c
 status(Q) = ready;
 enter(Q, ready-list);
 }
}

```

ThS. GVC Tô Oai Hùng

129

129

## Sử Dụng Monitor

- ❖ Với mỗi nhóm tài nguyên cần chia sẻ, định nghĩa một monitor, các thao tác trên tài nguyên này phải thỏa điều kiện nào đó.
- ❖ Các tiến trình muốn sử dụng tài nguyên chung, chỉ có thể thao tác thông qua các thủ tục bên trong monitor được gắn kết với tài nguyên:

```

Pi:
while (TRUE)
{
 Noncritical-section ();
 <monitor>.Procedurei(); // CS();
 Noncritical-section ();
}

```

ThS. GVC Tô Oai Hùng

130

130

## Sử Dụng Monitor

### ❖ Nhận xét:

- Đảm bảo truy xuất độc quyền bởi trình biên dịch mà không do lập trình viên → nguy cơ thực hiện đồng bộ hóa sai giảm rất nhiều.
- Đòi hỏi phải có một ngôn ngữ lập trình định nghĩa khái niệm monitor.

ThS. GVC Tô Oai Hùng

131

131

## Monitor và Producer/Consumer Problem

```

monitor ProducerConsumer
condition full, empty;
int count;
void insert(int item) {
 if (count == N)
 wait(full);
 insert_item(item);
 count ++;
 if (count == 1)
 signal(empty);
}
void remove(int &item); {
 if (count == 0)
 wait(empty);
 item = remove_item();
 count --;
 if (count == N-1)

```

// nếu bộ đệm đầy, phải chờ  
// đặt dữ liệu vào bộ đệm  
// tăng số chỗ đầy  
// nếu bộ đệm không trống  
// thì kích hoạt Consumer

// nếu bộ đệm trống, chờ  
// lấy dữ liệu từ bộ đệm  
// giảm số chỗ đầy  
// nếu bộ đệm không đầy

ThS. GVC Tô Oai Hùng

132

132

## Monitor và Producer/Consumer Problem

```

 signal(full);
 }
 count = 0;
end monitor;
void producer();
{
 while (TRUE)
 {
 item = produce_item();
 ProducerConsumer.insert(item);
 }
}
void consumer();
{
 while (TRUE)
 {
 ProducerConsumer.remove(item);
 consume_item(item);
 }
}

```

// thì kích hoạt Producer

**Figure 2-34.** An outline of the producer-consumer problem with monitors. Only one monitor procedure at a time is active. The buffer has  $N$  slots.

ThS. GVC Tô Oai Hùng

133

133

## Message

- ❖ Đồng bộ hóa trên môi trường phân tán.
- ❖ Được hỗ trợ bởi HĐH.
- ❖ Một tiến trình kiểm soát việc sử dụng tài nguyên và nhiều tiến trình khác yêu cầu tài nguyên.
- ❖ Tiến trình có yêu cầu tài nguyên:
  - Gởi một message đến tiến trình kiểm soát.
  - → chuyển sang trạng thái blocked cho đến khi nhận được thông điệp đánh thức từ tiến trình kiểm soát.
  - Khi sử dụng xong tài nguyên → gởi một thông điệp khác đến tiến trình kiểm soát để báo kết thúc truy xuất.

ThS. GVC Tô Oai Hùng

134

134

## Message

- ❖ Tiến trình kiểm soát:
  - Nhận được message yêu cầu tài nguyên.
  - Khi tài nguyên sẵn sàng → gửi một thông điệp đến tiến trình đang bị khóa trên tài nguyên đó để đánh thức tiến trình này.
- ❖ Thủ tục:
  - Send(destination, message): Gửi thông điệp đến tiến trình destination.
  - Receive(source, message): Nhận thông điệp từ tiến trình source.

ThS. GVC Tô Oai Hùng

135

135

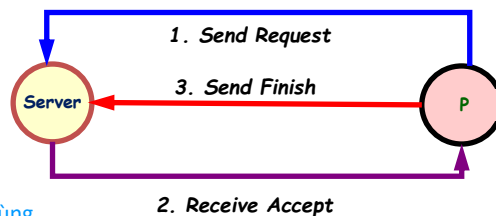
## Message

- ❖ Trao đổi thông điệp với hai thủ tục Send() và Receive() để thực hiện sự đồng bộ hóa:

```

while (TRUE)
{
 Send(process controler, request message);
 Receive(process controler, accept message);
 Critical-section();
 Send(process controler, end message);
 Noncritical-section();
}

```



ThS. GVC Tô Oai Hùng

136

136

## Các Bài Toán Đồng Bộ Hoá Kinh Điển

- ❖ **Producer – Consumer (Bounded-Buffer):**  
Người sản xuất – Người tiêu thụ.
- ❖ **Readers – Writers: Bộ Đọc - Bộ Ghi.**
- ❖ **Dinning Philosophers: 5 triết gia ăn tối.**

ThS. GVC Tô Oai Hùng

137

137

## 3.5 Điều Phối Tiến Trình

1. Mục tiêu điều phối.
2. Các cấp điều phối.
3. Các giải thuật điều phối (cấp điều phối thời gian ngắn).
4. Vấn đề điều phối luồng.

ThS. GVC Tô Oai Hùng

138

138

### 3.5.1 Mục Tiêu Điều Phối

#### ❖ Một số khái niệm:

1. Thời gian sử dụng CPU (*burst time/ execution time*): Khoảng thời gian mà tiến trình thực thi trên CPU.
2. Thời gian đến (*arrival time*): Thời gian mà tiến trình vào hàng đợi (vào trạng thái Ready) để sẵn sàng thực thi.
3. Thời gian đáp ứng (*response time*): Khoảng thời gian từ lúc tiến trình vào hàng đợi (ở trạng thái Ready) cho đến khi tiến trình được cấp CPU lần đầu tiên. Ví dụ:

ThS. GVC Tô Oai Hùng

139

139

### Mục Tiêu Điều Phối

| Tiến trình | Thời gian đến | Thời gian sử dụng CPU |
|------------|---------------|-----------------------|
| P1         | 0 ms          | 8 ms                  |
| P2         | 1 ms          | 7 ms                  |
| P3         | 2 ms          | 10 ms                 |

- P1: 0 ms.
- P2: 7 ms.
- P3: 13 ms.

#### Công thức:

- **Response time = Time at which the process gets the CPU for the first time - Arrival time.**

ThS. GVC Tô Oai Hùng

140

140

## Mục Tiêu Điều Phối

4. Thời gian chờ (*waiting time*): Tổng thời gian mà tiến trình chờ trong hàng đợi.

Công thức: Có 2 cách.

- $\text{Waiting time} = \text{Turnaround time} - \text{Burst time}.$
- $\text{Waiting time} = \text{Exit time} - \text{Arrival time} - \text{Burst time}.$

5. Thời gian hoàn thành (*turnaround time*): Khoảng thời gian từ lúc tiến trình vào hàng đợi đến khi tiến trình kết thúc.

Công thức: Có 2 cách.

- $\text{Turnaround time} = \text{Exit time} - \text{Arrival time}.$
- $\text{Turnaround time} = \text{Burst time} + \text{Waiting time}.$

ThS. GVC Tô Oai Hùng

141

141

## Mục Tiêu Điều Phối

### ❖ Mục tiêu chung:

- Công bằng sử dụng CPU, tận dụng CPU tối đa.
- Cân bằng sử dụng các thành phần của hệ thống.

### ❖ Hệ điều hành xử lý theo lô:

- Tối ưu thông lượng tối đa (throughput).
- Giảm thiểu turnaround time: Tquit – Tarrive.
- Tận dụng CPU.

### ❖ Hệ điều hành tương tác:

- Đáp ứng nhanh: Giảm thiểu thời gian chờ
- Cân đối mong muốn của người dùng.

ThS. GVC Tô Oai Hùng

142

142

## Mục Tiêu Điều Phối

- ❖ Hệ điều hành thời gian thực:
  - Tránh mất dữ liệu.
  - Đảm bảo chất lượng trong các hệ thống đa phương tiện.

ThS. GVC Tô Oai Hùng

143

143

## Mục Tiêu Điều Phối

- ❖ Tiêu chí lựa chọn tiến trình.
  - Chọn tiến trình vào RQ trước.
  - Chọn tiến trình có độ ưu tiên cao hơn.
- ❖ Thời điểm lựa chọn tiến trình.
  - Điều phối độc quyền (non-preemptive scheduling): tiến trình đang ở trạng thái Running sẽ tiếp tục sử dụng CPU cho đến khi kết thúc hoặc bị block vì chờ I/O hay các dịch vụ của hệ thống (độc chiếm CPU).
  - Điều phối không độc quyền (preemptive scheduling): tiến trình đang running phải trả CPU khi:

ThS. GVC Tô Oai Hùng

144

144



## Mục Tiêu Điều Phối

- Xử lý xong (kết thúc).
- Bị block chờ I/O hay các dịch vụ của hệ thống.
- Hết thời gian sử dụng CPU.
- Có tiến trình có độ ưu tiên hơn vào ReadyQueue.

ThS. GVC Tô Oai Hùng

145

145

## 3.5.2 Các Giải Thuật Điều Phối

- ❖ Điều phối trong Hệ điều hành xử lý theo lô (*Batch Systems*).
  - First-Come, First-Served (FCFS).
  - Shortest Job First (SJF).
  - Shortest Remaining Time Next (SRTF).
- ❖ Điều phối trong Hệ điều hành tương tác (*Interactive Systems*)
  - Round Robin.
  - Priority.
  - HRRN.
  - Multiple Queues.

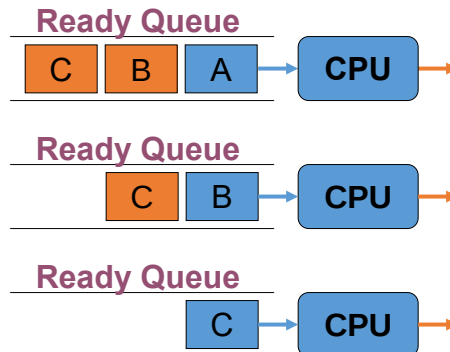
ThS. GVC Tô Oai Hùng

146

146

## First-Come First-Served (FCFS)

- ❖ Tiến trình vào hàng đợi (ready queue) trước sẽ nhận CPU trước.
- ❖ Thời điểm lựa chọn tiến trình.
  - Độc quyền.



ThS. GVC Tô Oai Hùng

147

147

## First-Come First-Served

- ❖ Ví dụ:

| P  | Arrival Time | Service Time |
|----|--------------|--------------|
| P1 | 0            | 24           |
| P2 | 1            | 3            |
| P3 | 2            | 3            |

- ❖ Biểu đồ Gantt:

|    |    |       |
|----|----|-------|
| P1 | P2 | P3    |
| 0  | 24 | 27 30 |

Thời gian đáp ứng:

$$P1: 0 - 0 = 0$$

$$P2: 24 - 1 = 23$$

$$P3: 27 - 2 = 25$$

$$\text{Avg} = (23 + 25) / 3 = 16$$

Thời gian hoàn thành:

$$P1: 24 - 0 = 24$$

$$P2: 27 - 1 = 26$$

$$P3: 30 - 2 = 28$$

$$\text{Avg} = (24 + 26 + 28) / 3 = 26.3$$

Thời gian chờ:

$$P1: 24 - 0 - 24 = 0$$

$$P2: 27 - 1 - 3 = 23$$

$$P3: 30 - 2 - 3 = 25$$

$$\text{Avg} = (23 + 25) / 3 = 16$$

148

148

## First-Come First-Served

- ❖ Đơn giản, dễ cài đặt.
- ❖ Chịu đựng hiện tượng tích lũy thời gian chờ
- ❖ Tiến trình có thời gian xử lý ngắn đợi tiến trình có thời gian xử lý dài.
- ❖ Ưu tiên tiến trình cpu-bounded.
- ❖ Có thể xảy ra tình trạng độc chiếm CPU.
- ❖ Không phù hợp với các hệ phân chia thời gian.

ThS. GVC Tô Oai Hùng

149

149

## Shortest Job First (SJF)

- ❖ Chọn tiến trình có thời gian xử lý ngắn nhất.
- ❖ Thời điểm lựa chọn tiến trình: độc quyền (non-preemptive).
- ❖ Ví dụ:

| P  | Arrival Time | Service Time |
|----|--------------|--------------|
| P1 | 0            | 6            |
| P2 | 1            | 8            |
| P3 | 2            | 4            |
| P4 | 3            | 2            |

Thời điểm lựa chọn độc quyền:

|    |    |    |    |
|----|----|----|----|
| P1 | P4 | P3 | P2 |
|----|----|----|----|

ThS. GVC Tô Oai Hùng

0      6      8      12      20      150

150

## Shortest Remaining Time Next (SRTN)

- ❖ Chọn tiến trình có thời gian xử lý còn lại ngắn nhất.
- ❖ Thời điểm lựa chọn tiến trình: Không độc quyền (khi có tiến trình mới vào RQ, thời gian xử lý còn lại của các tiến trình được tính toán lại và tiến trình có thời gian xử lý còn lại ngắn nhất sẽ được nhận CPU).

| P  | Arrival Time | Service Time |
|----|--------------|--------------|
| P1 | 0            | 6            |
| P2 | 1            | 8            |
| P3 | 2            | 4            |
| P4 | 3            | 2            |

- ❖ Thời điểm lựa chọn không độc quyền:

ThS. GVC Tô Oai

|    |    |    |    |    |    |
|----|----|----|----|----|----|
| P1 | P4 | P1 | P3 | P2 |    |
| 0  | 3  | 5  | 8  | 12 | 20 |

ThS. GVC Tô Oai

151

151

## Nhận Xét SJF và SRTN

- ❖ Nhận xét:
  - Tối ưu thời gian chờ.
  - Cơ chế không độc quyền có thể xảy ra tình trạng "đói" (starvation) đối với các process có CPU-burst lớn khi có nhiều process với CPU-burst nhỏ đến hệ thống.
  - Cơ chế độc quyền không phù hợp cho hệ thống chia sẻ thời gian.
  - Thời gian sử dụng CPU còn lại cho lần tiếp theo của mỗi tiến trình được tính theo công thức:

$$\tau_{n+1} = \alpha t_n + (1 - \alpha)\tau_n$$

ThS. GVC Tô Oai Hùng

152

152

## Nhận Xét SJF và SRTN

- Trong đó:
  - $t_n$  là độ dài của thời gian xử lý lần thứ  $n$ .
  - $t_{n+1}$  là giá trị dự đoán cho lần xử lý tiếp theo.
  - $0 \leq \alpha \leq 1$ .

ThS. GVC Tô Oai Hùng

153

153

## Điều Phối với Độ Ưu Tiên (Priority)

- ❖ Mỗi tiến trình được gán một độ ưu tiên để phân biệt tiến trình quan trọng với tiến trình bình thường.
- ❖ Tiêu chí lựa chọn tiến trình.
  - Tiến trình có độ ưu tiên cao nhất.
- ❖ Thời điểm lựa chọn tiến trình.
  - Độc quyền.
  - Không độc quyền.

ThS. GVC Tô Oai Hùng

154

154

## Điều Phối với Độ Ưu Tiên

❖ Ví dụ:

| P  | Arrival Time | Priority | Service Time |
|----|--------------|----------|--------------|
| P1 | 0            | 3        | 24           |
| P2 | 1            | 1        | 3            |
| P3 | 2            | 2        | 3            |

- Độ ưu tiên độc quyền:

|    |    |       |
|----|----|-------|
| P1 | P2 | P3    |
| 0  | 24 | 27 30 |

- Độ ưu tiên không độc quyền:

|    |    |    |      |
|----|----|----|------|
| P1 | P2 | P3 | P1   |
| 0  | 1  | 4  | 7 30 |

155

155

## Điều Phối với Độ Ưu Tiên

❖ Nhận xét:

- Có thể xảy ra tình trạng Starvation (đối CPU): các tiến trình độ ưu tiên thấp có thể không bao giờ thực thi được
- → Giải pháp Aging – tăng độ ưu tiên cho tiến trình chờ lâu trong hệ thống
- Gán độ ưu tiên còn dựa vào:
  - Yêu cầu về bộ nhớ.
  - Số lượng file được mở.
  - Tỷ lệ thời gian dùng cho I/O trên thời gian sử dụng CPU.
  - Các yêu cầu bên ngoài.

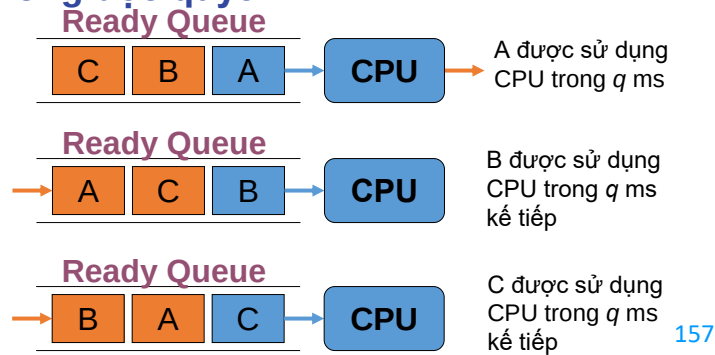
ThS. GVC Tô Oai Hùng

156

156

## Round Robin (RR) (Điều Phối Xoay Vòng)

- ❖ Mỗi tiến trình chỉ sử dụng một lượng  $q$  (quantum) cho mỗi lần sử dụng CPU.
- ❖ Tiêu chí lựa chọn tiến trình.
  - Thứ tự vào hàng đợi Ready Queue.
- ❖ Thời điểm lựa chọn tiến trình.
  - Không độc quyền.



157

## Round Robin

- ❖ Ví dụ:
  - RR ( $q=4$ ).

| P  | Arrival Time | Service Time |
|----|--------------|--------------|
| P1 | 0            | 24           |
| P2 | 1            | 3            |
| P3 | 2            | 3            |

- Biểu đồ Gantt:

|    |    |    |    |    |    |    |       |
|----|----|----|----|----|----|----|-------|
| P1 | P2 | P3 | P1 | P1 | P1 | P1 | P1    |
| 0  | 4  | 7  | 10 | 14 | 18 | 22 | 26 30 |

ThS. GVC Tô Oai Hùng

158

158

## Round Robin

### ❖ Nhận xét:

- Loại bỏ hiện tượng độc chiếm CPU.
- Phù hợp với hệ thống tương tác người dùng.
- Thời gian chờ đợi trung bình của giải thuật RR thường khá lớn nhưng thời gian đáp ứng nhỏ.
- Hiệu quả phụ thuộc vào việc lựa chọn quantum  $q$ .
  - $q$  quá lớn  $\Rightarrow$  FCFS (giảm tính tương tác)
  - $q$  quá nhỏ  $\Rightarrow$  chủ yếu thực hiện chuyển đổi ngữ cảnh (context switching).
  - Thường  $q = 10 - 100$  milliseconds.

ThS. GVC Tô Oai Hùng

159

159

## Highest Response Ratio Next (HRRN)

- ❖ Cải tiến giải thuật SJF.
- ❖ Định thời theo chế độ độc quyền.
- ❖ Độ ưu tiên được tính theo công thức:
 
$$p = (tw + ts) / ts$$
  - $tw$ : thời gian chờ (waiting time).
  - $ts$ : thời gian sử dụng CPU (service time).
- ❖ Tiêu chí chọn: chọn  $p$  lớn nhất  $\rightarrow$  Ưu tiên cho các tiến trình có thời gian sử dụng CPU ngắn và các tiến trình chờ lâu.
- ❖  $p$  được tính lại khi có tiến trình kết thúc.

ThS. GVC Tô Oai Hùng

160

160



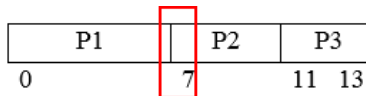
## Highest Response Ratio Next

❖ Ví dụ:

| P  | Thời gian đến | Thời gian sử dụng CPU |
|----|---------------|-----------------------|
| P1 | 0             | 7                     |
| P2 | 1             | 4                     |
| P3 | 5             | 2                     |

❖ Độ ưu tiên tại thời điểm (7):

- P2:  $(6+4)/4 = 2.5$
- P3:  $(2+2)/2 = 2$
- → chọn P2



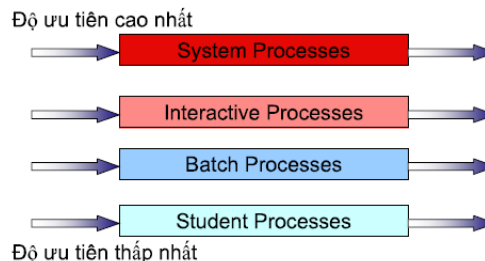
ThS. GVC Tô Oai Hùng

161

161

## Định Thời Hàng Đợi Nhiều Mức (Multilevel Queue Scheduling)

- ❖ Sử dụng nhiều hàng đợi ready (ví dụ foreground, background).
- ❖ Mỗi hàng đợi chứa các tiến trình có cùng độ ưu tiên và có giải thuật điều phối riêng, ví dụ:
  - foreground: RR
  - background: FCFS
- ❖ HĐH phải định thời cho các hàng đợi
- ❖ Ví dụ:



ThS. GVC Tô Oai Hùng

162

162

## Multilevel Queue Scheduling

- ❖ Fixed priority scheduling:
  - Chọn tiến trình trong hàng đợi có độ ưu tiên từ cao đến thấp.
  - Có thể có starvation.
- ❖ Time slice:
  - Mỗi hàng đợi được nhận một khoảng thời gian chiếm CPU và phân phối cho các process trong hàng đợi khoảng thời gian đó.
  - Ví dụ: 80% cho hàng đợi foreground định thời bằng RR và 20% cho hàng đợi background định thời bằng giải thuật FCFS.

ThS. GVC Tô Oai Hùng

163

163

## Hàng Đợi Phản Hồi Nhiều Mức (Multilevel Feedback Queue)

- ❖ Trong hệ thống Multilevel Feedback Queue, các tiến trình có thể được di chuyển giữa các queue (tự động chuyển đổi mức độ ưu tiên) tùy theo đặc tính của nó tại mỗi thời điểm.
- ❖ Ví dụ:
  - Tiến trình sử dụng CPU quá lâu → chuyển sang một hàng đợi có độ ưu tiên thấp hơn.
  - Tiến trình chờ quá lâu trong một hàng đợi có độ ưu tiên thấp → chuyển lên hàng đợi có độ ưu tiên cao hơn (aging, giúp tránh starvation).

ThS. GVC Tô Oai Hùng

164

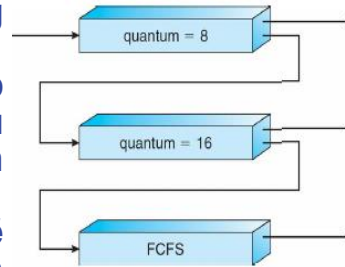
164

## Multilevel Feedback Queue

- ❖ Ví dụ: Có 3 hàng đợi:
  - Q0 dùng RR với quantum 8 ms.
  - Q1 dùng RR với quantum 16 ms.
  - Q2 dùng FCFS.

### ❖ Giải thuật:

- Tiến trình mới sẽ vào hàng đợi Q0.
- Khi đến lượt, sẽ được cấp phát quantum là 8 ms, nếu chưa xử lý xong → chuyển xuống cuối hàng đợi Q1.
- Tại Q1, tiến trình thực thi sẽ được cấp phát quantum là 16 ms, nếu chưa xử lý xong → chuyển xuống cuối hàng đợi Q2.



ThS. GVC Tô Oai Hùng

165

165

## 3.5.3 Vấn Đề Điều Phối Luồng

- ❖ Các luồng liên lạc với nhau thông qua các biến toàn cục của tiến trình.
- ❖ Cơ chế điều phối luồng phụ thuộc vào cách cài đặt luồng:
  - Cài đặt trong user-space.
  - Cài đặt trong kernel-space.
  - Cài đặt trong kernel-space và user-space.

ThS. GVC Tô Oai Hùng

166

166

## Vấn Đề Điều Phối Luồng

### ❖ Cài đặt trong user-space:

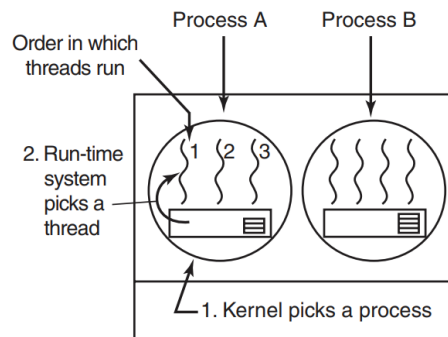
- Bảng quản lý thread lưu ở phần user-space.
- Tiến trình chịu trách nhiệm điều phối thread.
- Kernel không biết được sự tồn tại của luồng.
- Giả sử quantum của process = 50 msec, quantum của thread = 5 msec và giả sử tiến trình A có ba thread, tiến trình B có 4 thread.
- Thứ tự điều phối có thể là A1, A2, A3, A1, A2, A3, A1, trước khi kernel chuyển sang tiến trình B.

ThS. GVC Tô Oai Hùng

167

167

## Vấn Đề Điều Phối Luồng



Possible: A1, A2, A3, A1, A2, A3

Not possible: A1, B1, A2, B2, A3, B3

- Không thể A1 đến B1 được do thread của tiến trình nào tiến trình đó quản lý. Và tiến trình A chưa hết quantum nên thread của tiến trình B không thể thực hiện.

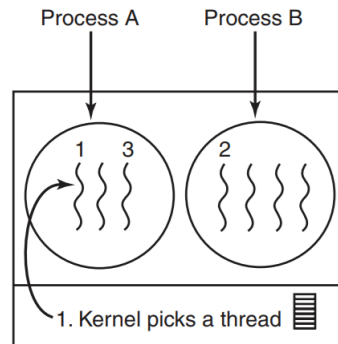
ThS. GVC Tô Oai Hùng

168

168

## Vấn Đề Điều Phối Luồng

- ❖ Cài đặt trong kernel-space:
  - Bảng quản lý thread lưu ở phần kernel-space.
  - HĐH chịu trách nhiệm điều phối thread.



ThS. GVC Tô Oai Hùng

Possible: A1, A2, A3, A1, A2, A3  
 Also possible: A1, B1, A2, B2, A3, B3

169

169

## Vấn Đề Điều Phối Luồng

- Thứ tự điều phối có thể là A1, B1, A2, B2, A3, B3, ... vì các thread do hệ điều hành quản lý.
- Sự khác biệt chính giữa các luồng trong user-space và kernel-space là hiệu suất.
- Việc chuyển một luồng trong tiến trình A sang một luồng trong tiến trình B đắt hơn việc chạy luồng thứ hai trong tiến trình A.

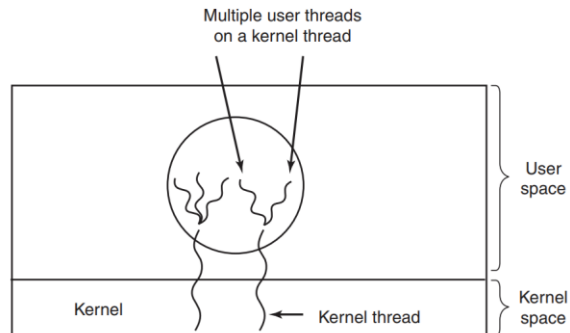
ThS. GVC Tô Oai Hùng

170

170

## Vấn Đề Điều Phối Luồng

- ❖ Cài đặt trong kernel-space và user-space:
  - Một số luồng mức user được cài đặt bằng một luồng mức kernel.
  - Một luồng của HĐH quản lý một số luồng của tiến trình.



Multiplexing user-level threads onto kernel-level threads.

ThS. GVC Tô Oai Hùng

171